
PINIT ECOSYSTEM

Product & Development Completion Documentation

Pinit HUB & Pinit Exchange

Document purpose	Official development-status record of the functionality that has been fully implemented in Pinit HUB and Pinit Exchange
Version	1.0
Documentation date	21 September 2026
Prepared by	Pinit Development Team
Products	Pinit HUB · Pinit Exchange

How to Read This Document

This is an implementation-status document. It describes what has been built in the Pinit HUB and Pinit Exchange repository and what evidence exists that it works. It is not a roadmap and it is not a marketing description.

Definition of COMPLETED

A capability is listed as completed only when all of the following hold:

- **Implemented end to end:** a frontend screen, a backend route and service, and (where data is stored) a database entity all exist and are connected.
- **Evidenced:** either automated tests for the capability pass **[T]**, or the capability has demonstrably run against the production database, leaving real records **[O]**.
- **No known failing tests:** a capability whose own automated tests currently fail is not presented as completed, even if a screen exists.

Evidence key

Code	Meaning
[T]	Automated test suites covering the capability pass in the reviewed tree.
[O]	Real records created by the capability exist in the production database (row counts taken read-only on the documentation date).
[C]	Frontend, backend and schema were reviewed together in the repository and are consistent.

Repository baseline

Item	Value
Repository	Pinit-DNA (Hub backend, Hub client, Python AI service, Exchange app and server)
Branch / latest commit	ashwitha / aa0cacd
Hub schema	Prisma, 111 models at the committed baseline, PostgreSQL (Supabase)
Exchange schema	21 tables, PostgreSQL schema 'exchange' in production; SQLite for local development
Review date	21 September 2026

Two disclosures that shape the scope

Uncommitted work. Several capabilities were developed and verified locally but are not yet committed or deployed, so they are excluded from the completed scope (see “Not Included in Completed Scope”).

Test status. Where a module’s own tests fail in the reviewed tree, the module is excluded and the failing suites are named. Section 26 reports the full test status without rounding.

Contents

1	Executive Summary
2	Product Vision
3	Pinit Ecosystem Overview
4	Pinit HUB: Product Overview
5	Pinit HUB: Complete User Flow
6	Asset Protection Workflow
7	Pinit DNA Architecture
8	Vault and Asset Architecture
9	Database Architecture (HUB)
10	Backend Architecture
11	API Architecture
12	Security and Authentication
13	Sharing and Access Control
14	Certificate and Verification System
15	Monitoring and Intelligence
16	Business Workspace
17	Pinit Exchange: Product Overview
18	Exchange Seller Workflow
19	Exchange Buyer Workflow
20	Exchange Database Architecture
21	HUB + Exchange Integration
22	Frontend Architecture
23	File and Project Structure
24	End-to-End Data Flow
25	Completed Development Inventory
26	Testing and Validation
27	Deployment Architecture
28	User Journeys
29	Technical Entity Map
30	Completed Product Capabilities
31	Final Development Status
	Not Included in Completed Scope
	Documentation Scope

1. Executive Summary

Pinit is a connected pair of products for protecting, proving, sharing and trading digital creative work.

Pinit HUB

Pinit HUB is where a digital asset is protected, identified and managed. When a file is protected, HUB computes its **Pinit DNA**, stores an **encrypted copy in the Vault**, records the file as an **Asset** with a canonical identifier, and can issue a signed **Certificate** that anyone can verify. The owner can then share the asset through controlled **Secure Share** links, see who opened them, and enrol the asset in **Monitoring**. HUB also provides a Business Workspace (clients, campaigns, people and review workflows) and a Portfolio.

- Digital asset protection and asset identity (Pinit DNA, Layers 1 to 6)
- Secure, encrypted asset management in the Vault
- Ownership and provenance records linking file, DNA, vault copy and certificate
- Controlled sharing with access controls, activity records and revocation
- Certificate issue, public verification and revocation
- Monitoring of protected assets against supported public sources
- Business Workspace, Portfolio, notifications and plans

Pinit Exchange

Pinit Exchange is the marketplace. A creator lists a protected asset from HUB, a buyer discovers it, pays, and receives a licence; the seller sees the sale and the earnings. Exchange does not hold the master identity or the vault: HUB remains the source of the asset and its protection, and the two products are joined by a verified bridge.

- Marketplace discovery, listing detail, cart and wishlist
- Seller onboarding, listing from a HUB asset, sales and earnings
- Buyer checkout with Razorpay, sealed licence records, invoices and licensed delivery
- Opportunities: briefs, proposals, invitations and awards between buyers and creators

How the two fit together

Pinit HUB = protect, manage, verify and monitor digital assets.

Pinit Exchange = discover, list, buy, sell and license digital assets.

Together they form one asset lifecycle: an asset protected in HUB is the same asset that is listed, sold and delivered through Exchange.

2. Product Vision

Pinit treats a digital asset as something with an identity, a proof of origin and a controlled life, rather than as a file in a folder.

Protection lifecycle (implemented)

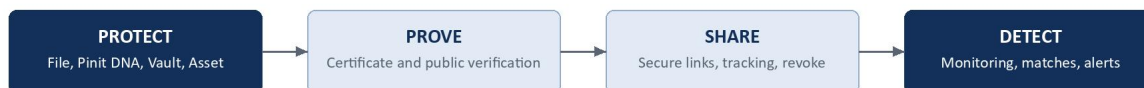


Figure 1. Protection lifecycle in Pinit HUB

The fifth stage of the originally intended lifecycle, **Investigate**, is not part of the completed scope; see “Not Included in Completed Scope”.

Commercial lifecycle (implemented)



Figure 2. Commercial lifecycle across HUB and Exchange

What is different from ordinary storage

Concept	Ordinary file storage	Pinit
Digital asset identity	A filename and a path	A canonical Asset.id , distinct from the vault, DNA and certificate identifiers
Protection	Access control on a folder	File bytes encrypted with AES-256-GCM in the Vault; an ownership watermark carried in the file
Provenance	Modified-time metadata	Pinit DNA : hash, structural, perceptual, colour and metadata layers recorded at protection time
Proof	None	A signed Certificate with a public verification link; status can be ACTIVE, REVOKED or EXPIRED
Controlled access	Shared folder permissions	Secure Share links with expiry, view and download limits, one-time use, OTP, country/device/IP rules, viewer blocking and revocation
Marketplace transactions	External to the file	Listings and licences in Exchange tied to the same Asset.id, with sealed order records

3. Pinit Ecosystem Overview

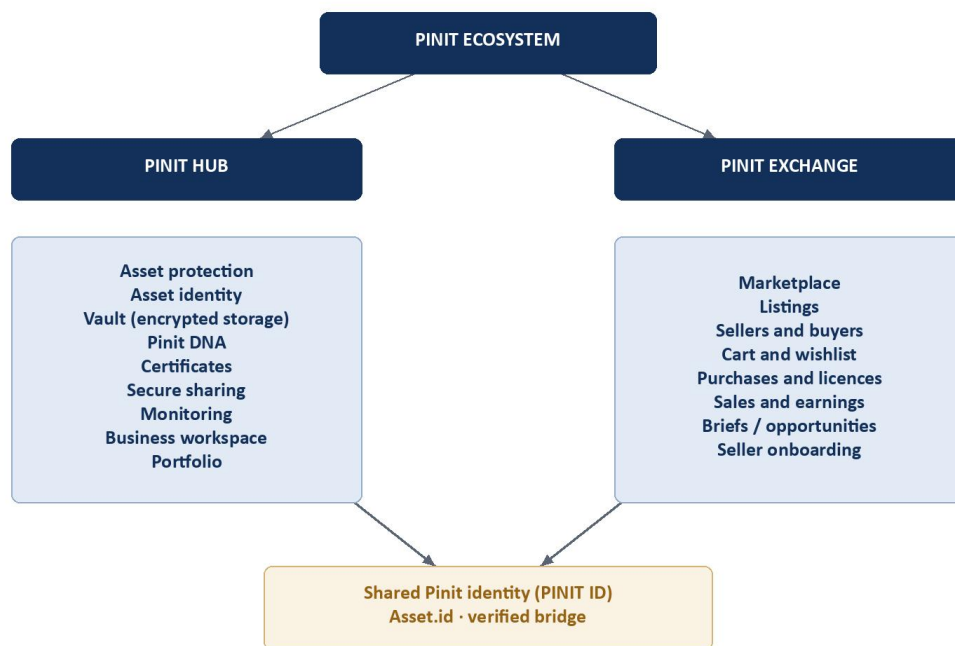


Figure 3. Pinit ecosystem architecture

Only relationships that exist in the implementation are drawn. The two products are separate applications with separate servers and separate schemas. They are joined by:

- **Shared Pinit identity:** the HUB user's public PINIT ID is the identity Exchange uses for sellers and buyers.
- **Asset.id:** the canonical HUB asset identifier is the value Exchange stores as the asset of a listing.
- **A verified bridge:** HUB-to-Exchange calls carry a bridge secret; Exchange receives only public-safe listing data and short-lived sign-in tokens. Vault keys and full forensic DNA never leave HUB.

4. Pinit HUB: Product Overview

Each module below is completed by the definition on page 2. Modules that do not meet it are listed in the final section.

4.1 Authentication and Identity

Purpose	Create a Pinit account and sign in. Authentication proves who is signing in; it is never used as ownership.
User workflow	Create account, then sign in with password, face verification or a passkey. The session is an access token (Bearer JWT) plus an HttpOnly refresh cookie; the user can refresh or log out.
Frontend	pages/auth, components/auth (FaceRoundScan, BiometricStep, CameraStage), context/AuthContext, components/auth/RequireAuth, lib/webauthn, lib/face-liveness
Backend / API	POST /auth/create, /login, /refresh, /logout; GET /auth/me; /auth/face/{challenge, register, login, identify, status}; /auth/passkey/{register, login}/{start, finish}. Middleware requireAuth; lib/auth-cookies.
Database	User, RefreshToken, BiometricIdentity, FaceTemplate, WebAuthnCredential, LoginHistory, UserSession, UserDevice, SecurityEvent
Completed	Password accounts, JWT access and refresh handling, face enrolment and login with liveness checks, passkey enrolment and login, session listing and revocation.
Evidence	[T] auth suites: auth-cookies, face-liveness-pad, face-match-gate, face-verify-claimed, pad-face-presence, passkey-enroll-gate, webauthn-passkey, webauthn-ownership-fix, and the integration suites (JWT subject and isolation, 1:1 login, duplicate registration, no embedding in logs or responses). [O] 7 face templates recorded. No passkey enrolment is recorded in production data; passkeys are evidenced by tests only.

4.2 Personal and Business Accounts

Purpose	Let one identity work as a personal creator or as a business owner with a workspace.
User workflow	Choose an account type, complete business setup, switch between personal and business views.
Frontend	components/nav/WorkspaceSwitcher, context/AccountViewModeContext, hooks/useSyncBusinessSetup, layouts/DashboardLayout
Backend / API	POST /auth/account-type, POST /auth/business-setup, GET /auth/business-setup/status, POST /organization/setup, GET /organization/me
Database	Organization, Workspace, OrganizationMember, Department
Completed	Account-type selection, business setup, workspace creation and view switching.
Evidence	[T] login-workspace-default. [O] 4 organizations.

4.3 Protect New

Purpose	Turn a file into a protected asset in one guided flow.
User workflow	Upload a file, watch the six DNA layers run, encrypt and store in the Vault, then receive the certificate. Detail in Section 6.
Frontend	pages/GeneratePage (wraps App.tsx), UploadZone, LayerPipeline, LayerCard, GenerationProgress, EncryptionStep, VaultStep, ProtectReadyStep, SuccessPanel

Backend / API	POST /dna/generate, POST /vault/store, POST /certificates
Database	DnaRecord and layer records, VaultRecord, Asset, Certificate
Completed	The whole flow from upload to a stored, identified, certified asset.
Evidence	[T] duplicate-check, duplicate-unowned-records, asset-identity-chain, asset-linkage-phase1. [O] 42 assets, 42 vault records, 32 certificates and 575 DNA records (which include records generated internally for video frames).

4.4 Vault, My Assets and Asset Details

Purpose	Keep the protected files and manage them.
User workflow	Open My Assets, filter and search, open an asset for preview, details, sharing, tracking and certificate; rename, retrieve, download a protected copy or remove the stored file.
Frontend	pages/VaultPage, pages/assets/AssetsPage, pages/assets/AssetDetailPage, components/VaultDetailSidePanel, VaultFileThumbnail
Backend / API	GET /vault/, GET /vault/:id, GET /vault/:id/preview, POST /vault/:id/retrieve, PATCH /vault/:id/rename, DELETE /vault/:id, POST /vault/:id/protected-download/prepare and /protected-download, GET /vault/:id/tracking; GET /assets, GET /assets/:id
Database	VaultRecord, Asset, TrackedExportPackage
Completed	Encrypted storage and retrieval, preview, rename, protected download packages, removal that keeps the DNA record and the asset record.
Evidence	[T] vault suites: owner-binding, protected-download, vault-blob-integrity, vault-delete-asset-cleanup, asset-identity-chain, asset-linkage-phase1; services/vault-storage-path, vault-preview-bytes. [O] 42 vault records.

4.5 Security Check

Purpose	Confirm that stored files are present and uncorrupted.
User workflow	Run the integrity check; problems raise a notification.
Frontend	pages/VaultIntegrityPage
Backend / API	GET /vault/integrity-check
Database	VaultRecord, PlatformEvent
Completed	Integrity verification of stored vault files, with issues reported as events.
Evidence	[T] vault-blob-integrity. [O] integrity-issue events recorded.

4.6 Certificates and Verification

Purpose	Give an asset a verifiable proof and let anyone check it.
User workflow	Issue a certificate from an asset, preview and download it, share the verification link, revoke it if needed. A third party opens the link or scans the QR code to check the status. Detail in Section 14.
Frontend	pages/CertificatesPage, pages/VerifyCertificatePage, components/certificates (CredentialPreviewModal, DownloadCertificateButton, ShareCertificateButton, PinitCredentialDocument)

Backend / API	POST /certificates, GET /certificates, GET /certificates/dna/:dnaRecordId, GET /certificates/verify/:certificateId (public), POST /certificates/revoke/:certificateId; GET /credentials/me
Database	Certificate, Credential
Completed	Issue (idempotent per DNA and vault pair), HMAC signature, verification with ACTIVE / REVOKED / EXPIRED / NOT_FOUND results, revocation with a reason, expiry handling, certificate document with QR code.
Evidence	[T] credentials-phase1 (certificate to credential projection, ownership isolation). [O] 32 certificates.

4.7 Secure Sharing and Link Intelligence

Purpose	Share a protected asset under the owner's control and see what happens to it.
User workflow	Choose an asset, set the controls, create the link, send it. The owner sees each open, download and blocked action, a map of access, and the chain of any onward sharing; the owner can block a viewer, force a logout or revoke the link. Detail in Section 13.
Frontend	pages/VaultSharePage, VaultShareManagePage, ShareViewerPage (public), AccessIntelligencePage, LinkIntelligencePage, LinkTreePage; components/share, maps
Backend / API	POST /share, POST /share/file, GET /share, GET /share/:token, POST /share/:token/access, /verify-otp, /share-further, GET /share/:token/logs, /tree, /export, DELETE /share/:token, block-viewer, force-logout; GET /share/analytics/geo and /live-map; /recipients
Database	ShareLink, ShareRecipient, ShareAccessLog, LinkForwardEvent, BlockedShareViewer, RecipientProfile
Completed	Link creation with access rules, recipient records, access logging with device and location, forwarding chains, viewer blocking, revocation, CSV export.
Evidence	[T] share-link-revoked-parent, share-unavailable-copy, share-viewer-url, authz-boundary, tenant-isolation. [O] 39 links, 195 access-log rows.

4.8 Monitoring

Purpose	Watch supported public sources for copies of a protected asset.
User workflow	Enrol assets, let scans run, review alerts, confirm or dismiss them. Detail in Section 15.
Frontend	pages/MonitoringPage
Backend / API	/monitor: enroll, enroll-all, list, alerts, alerts/:id/confirm and /dismiss, :id/check, :id/runs, pause, resume, watch-urls, scan-type, stats
Database	MonitorRecord, MonitoringRun, MonitoringFailure, CrawlerJob, CrawlResult, CrawlerMatch, AssetDiscovery
Completed	Enrolment, scheduled scanning through the crawler engine, result and match records, alerts with a confirm or dismiss decision.
Evidence	[O] 96 monitor records, 20,527 crawl results, 3,847 scan-completed events for the largest account. There is no dedicated automated test suite for the crawler; it is evidenced by operation only.

4.9 Business Workspace

Purpose	Let an agency or company organise protected assets by client and campaign, with people, review
----------------	--

	and handover.
User workflow	Business, clients, campaigns, people, assets, collaboration. Detail in Section 16.
Frontend	pages/business (BusinessDashboardPage, ClientsPage, ClientWorkspacePage, CampaignWorkspacePage, BusinessTeamPage, OrganizationSettingsPage, BusinessAuditLogsPage)
Backend / API	/business (clients, campaigns, members, people access, assets, versions, review, approvals, messages, handovers) and /organization (profile, team, invites, roles, workspaces, departments, audit logs)
Database	Organization, OrganizationMember, OrganizationInvite, Department, Workspace, Client, Campaign, CampaignMember, CampaignMemberAsset, AssetVersion, VersionApproval, ReviewComment, CampaignMessage, CampaignHandover, OrganizationAuditLog
Completed	Clients, campaigns, campaign members and per-person access, campaign assets and versions, review comments and approvals, campaign messages, handovers, team roles, departments, audit log.
Evidence	[T] campaign-people-team, share-review, share-message. [O] 4 organizations, 2 clients, 2 campaigns, 14 version-approved events.

4.10 Portfolio

Purpose	Present a creator's protected work publicly.
User workflow	Build the portfolio from protected assets, preview, publish or unpublish, share the public page.
Frontend	pages/ProfilePage (Portfolio tab), components for portfolio editing
Backend / API	GET and PUT /portfolio/me, POST /portfolio/me/publish and /unpublish, GET /portfolio/me/preview, GET /portfolio/public/:slug
Database	Portfolio, PortfolioProfile, PortfolioProject, PortfolioProjectMedia (and related section models)
Completed	Draft, preview, publish and unpublish with a frozen published snapshot; public page by slug.
Evidence	[T] portfolio-phase1. [O] 6 portfolios.

4.11 Profile, Settings and Notifications

Purpose	Account details, preferences, sessions and the notification inbox.
User workflow	Edit the profile and avatar, change the password, manage sessions, choose what to be notified about, read and archive notifications.
Frontend	pages/ProfilePage, components/nav (Topbar, notification bell)
Backend / API	/profile (profile, avatar, password, notifications, stats, activity, sessions); /notifications (stream, list, read, read-all, archive, delete)
Database	User, UserSession, Notification, PlatformEvent
Completed	Profile and session management; a notification inbox delivered over a server-sent event stream; platform events raised by protection, sharing, certificates and monitoring.
Evidence	[T] notification-inbox, notification-policy, historical-notification-link.

4.12 Plans and Entitlements

Purpose	Define what each account may do.
User workflow	View plans and current subscription; features are enabled or blocked by the plan.
Frontend	pages/UpgradePage, pages/subscription, hooks/useSubscription
Backend / API	GET /subscription/me, /plans, /billing/config, /billing/history; requireFeature middleware
Database	Plan, Subscription, FeatureEntitlement, UsageRecord, BillingHistory
Completed	Plan catalogue, subscription state, feature gating and storage-policy limits.
Evidence	[T] asset-upload-storage-policy, enterprise-feature-flags. [O] 7 subscriptions. Payment collection for Hub plans is outside the completed scope (see final section).

5. Pinit HUB: Complete User Flow

The table follows one asset from account creation to a shareable, monitored, verifiable asset. Every stage shown is implemented.

Step	What happens	Frontend	Backend / API	Records	Output
1 Account	User creates an account and receives a PINIT ID.	pages/auth	POST /auth/create	User	Account
2 Authenticate	Password, face or passkey sign-in.	pages/auth, components/auth	POST /auth/login, /face/login, /passkey/login/*	RefreshToken, UserSession, LoginHistory	Access JWT + refresh cookie
3 Dashboard	Home view for the account.	pages/DashboardPage, pages/home	GET /assets/stats, GET /profile/stats	reads	Summary of assets and activity
4 Protect New	File uploaded and validated.	GeneratePage, UploadZone	POST /dna/generate	DnaRecord	DNA record id
5 Analysis and DNA	Layers 1 to 6 computed.	LayerPipeline	dna.orchestrator, layers 1 to 6	Layer records	Pinit DNA
6 Vault	Encrypted copy stored.	VaultStep, EncryptionStep	POST /vault/store	VaultRecord + file in storage	Vault id
7 Asset identity	Asset record created, linked to DNA and vault.	(automatic)	assetService.ensureAssetFromProtect	Asset	Asset.id
8 Certificate	Signed certificate issued.	ProtectReadyStep, CertificatesPage	POST /certificates	Certificate	Certificate id
9 My Assets	Asset appears for management.	VaultPage, AssetsPage	GET /vault, GET /assets	reads	Managed asset
10 Share / Monitor	Owner shares or enrolls in monitoring.	VaultSharePage, MonitoringPage	POST /share, POST /monitor/enroll/:dnaRecordId	ShareLink, MonitorRecord	Link; monitoring profile
11 Verify	Anyone checks the certificate.	VerifyCertificatePage	GET /certificates/verify/:id	reads (and a lifecycle event)	ACTIVE / REVOKED / EXPIRED / NOT_FOUND

Not in this flow

Investigation and evidence reporting is excluded from the completed scope. Steps 9 to 11 are the end of the completed HUB flow.

6. Asset Protection Workflow

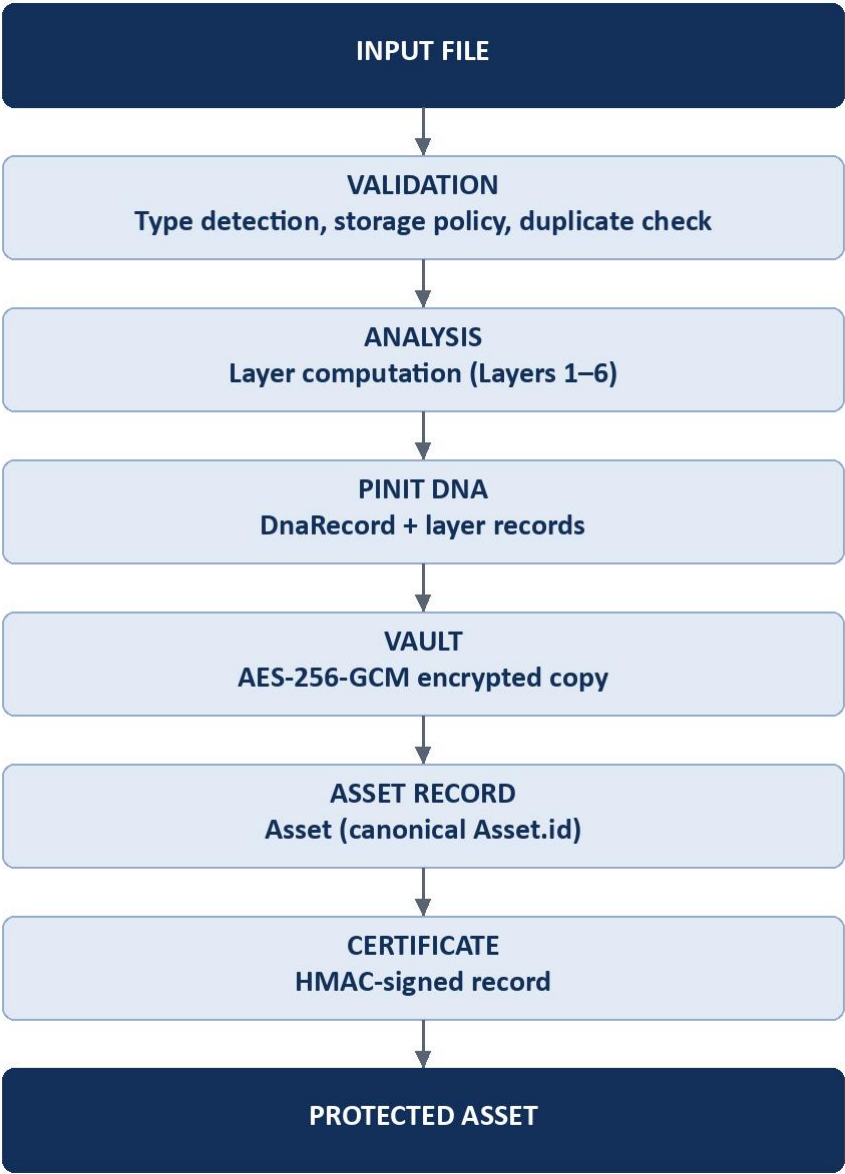


Figure 5. Asset protection flow

Stage	What happens	Verified in
Input file	The user selects or drops a file. Supported types are declared by the backend (GET /dna/supported-types).	UploadZone, dna.routes
Validation	File type detection, storage-policy limits for the plan, and a duplicate check that compares the file with existing records so that a file already owned by someone else is refused.	universal-file-router, duplicate-check.service, tests duplicate-check and duplicate-unowned-records
Analysis	Layers 1 to 6 are computed from the file bytes (Section 7). Some analysis is delegated to the Python AI service.	src/services/layers, python-ai
DNA	A DnaRecord is created with the layer results and status. Identity is deterministic for the same file content.	dna.orchestrator, deterministic-identity test
Vault	The file is encrypted with AES-256-GCM using a key derived per vault record and stored (Supabase Storage in production). A VaultRecord links it to the DNA record.	encryption.service, vault.service

Stage	What happens	Verified in
Asset record	An Asset row is created or reused, linked to the DNA record and the vault record, carrying the SHA-256 of the file. This is the canonical Asset.id.	assetService.ensureAssetFromProtect (vault.service)
Ownership and provenance	The owner is stamped on every record. The ownership watermark is embedded (Layer 6) and provenance events are appended.	layer6, forensic-provenance.service
Certificate	A certificate is issued for the DNA and vault pair and linked to the asset.	certificate.service
Availability	The asset appears in Vault / My Assets with its certificate, sharing and monitoring options.	VaultPage, AssetsPage

The order in which records are created is: **DnaRecord, then VaultRecord, then Asset, then Certificate**. The Asset row is the canonical identity but is created at vault storage, after the DNA record exists.

7. Pinit DNA Architecture

Pinit DNA is the recorded identity of a file: a set of independent characteristics computed from the file at protection time and stored against a DnaRecord. It exists so that a file can later be recognised, compared and proven, even after it has been re-saved, resized or partly altered.

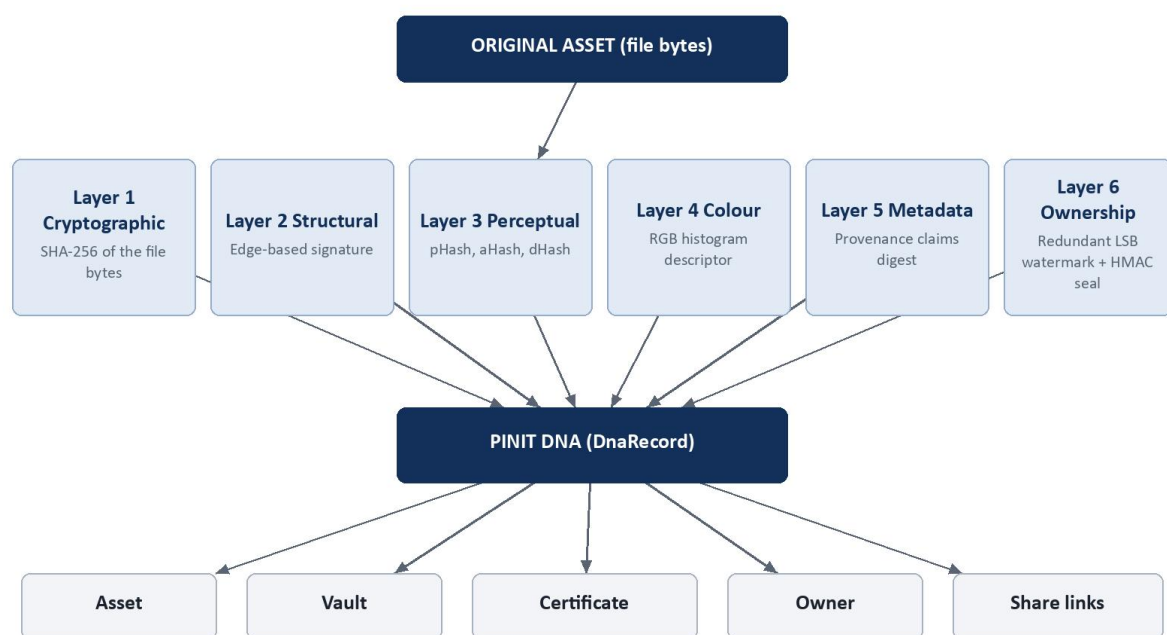


Figure 6. Pinit DNA architecture (implemented layers only)

Layer	What it records	Use	Tests
1 Cryptographic	SHA-256 of the raw file bytes (exact file identity)	Exact-match identity; part of the certificate and asset record	layer1.cryptographic
2 Structural	Edge-based structural signature	Recognising the same structure after mild change	layer2.structural
3 Perceptual	pHash, aHash and dHash of a reduced greyscale image	Recognising visually similar files	layer3.perceptual
4 Colour	RGB histogram descriptor	Colour profile of the file	layer4.semantic

Layer	What it records	Use	Tests
5 Metadata	Provenance claims digest	Deterministic identity from content and claims	layer5.metadata
6 Ownership watermark	Redundant least-significant-bit signature tiled across the image plus an HMAC identity seal over layers 1 to 5	Recovering ownership from a cropped copy	layer6.steganography, ownership-watermark

Relationships of a DnaRecord

- **Asset:** Asset.dnaId is a foreign key to the DnaRecord.
- **Vault:** VaultRecord.dnaRecordId is a foreign key (unique) to the DnaRecord.
- **Certificate:** Certificate.dnaRecordId holds the DNA record id; it is signed into the certificate.
- **Owner:** DnaRecord.ownerUserId references the User.
- **Sharing and monitoring:** ShareLink.dnaRecordId and MonitorRecord.dnaRecordId (foreign key) point back to it.

Scope of this section

Only Layers 1 to 6, which have implementation and passing tests, are documented. The schema also defines further layer tables and the codebase contains additional DNA modes; they are outside the verified scope.

8. Vault and Asset Architecture

These concepts are separate identifiers on purpose. **Asset.id** is the canonical identity used across HUB and Exchange. Vault, DNA and Certificate identifiers address the records they belong to and are never used in place of Asset.id.

Concept	Model	What it is	Linked to Asset by
User	User	The owner and the identity behind every record	Asset.ownerUserId (foreign key)
Asset	Asset	Canonical identity of a protected file; carries the file SHA-256, type, status, monitoring profile	(itself)
DNA record	DnaRecord	The Pinit DNA of the file	Asset.dnaId (foreign key)
Vault record	VaultRecord	The encrypted stored copy and its size and path	Asset.vaultId (value); VaultRecord.dnaRecordId (foreign key)
Certificate	Certificate	Signed proof; status ACTIVE, REVOKED, EXPIRED	Certificate.assetId (value), Asset.certificateId (value)
Protected file	file in Supabase Storage	The encrypted bytes, addressed by vault id	through VaultRecord
Activity	AssetTimelineEvent, PlatformEvent, ShareAccessLog	What happened to the asset	AssetTimelineEvent.assetId (foreign key)
Monitoring	MonitorRecord, CrawlResult	Watching for copies	MonitorRecord.dnaRecordId (foreign key); Asset.monitorRecordId (value)
Discovery	AssetDiscovery	A copy found by monitoring	AssetDiscovery.assetId (foreign key)

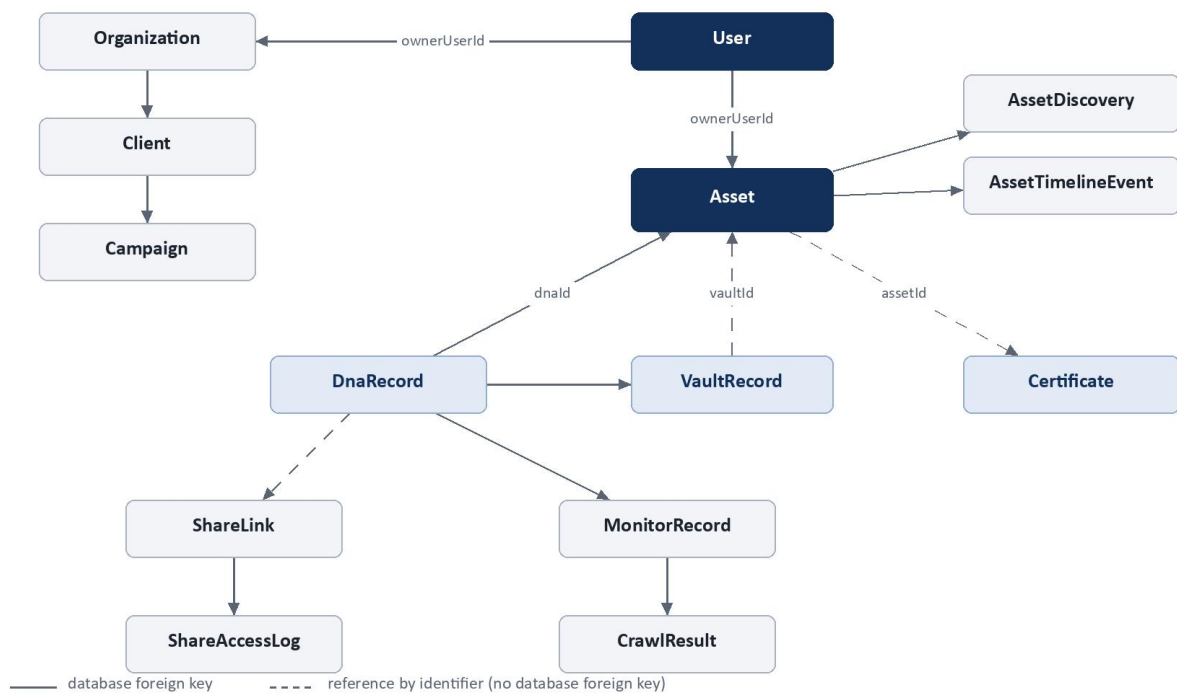


Figure 7. Asset and vault architecture (HUB entity relationships)

Reading the diagram

Solid lines are database foreign keys. Dashed lines are references by identifier that the application maintains but the database does not enforce (for example Asset.vaultId and Certificate.assetId). Arrows run from the referenced record to the record that holds the reference. Share links also carry the vault and DNA identifiers as plain values.

9. Database Architecture (HUB)

Item	Detail
Technology	PostgreSQL hosted on Supabase, accessed through Prisma 5 (@prisma/client)
Schema	prisma/schema.prisma: 111 models at the committed baseline; every model uses a UUID string primary key named id unless stated
Ownership model	Records carry ownerUserId; every protected API enforces that the JWT subject equals the resource owner (lib/tenant-scope)
Changes to the production schema	Applied by idempotent ensure-*.cjs scripts run at start-up (start:prod), not by migrations

A. High-level entity relationships

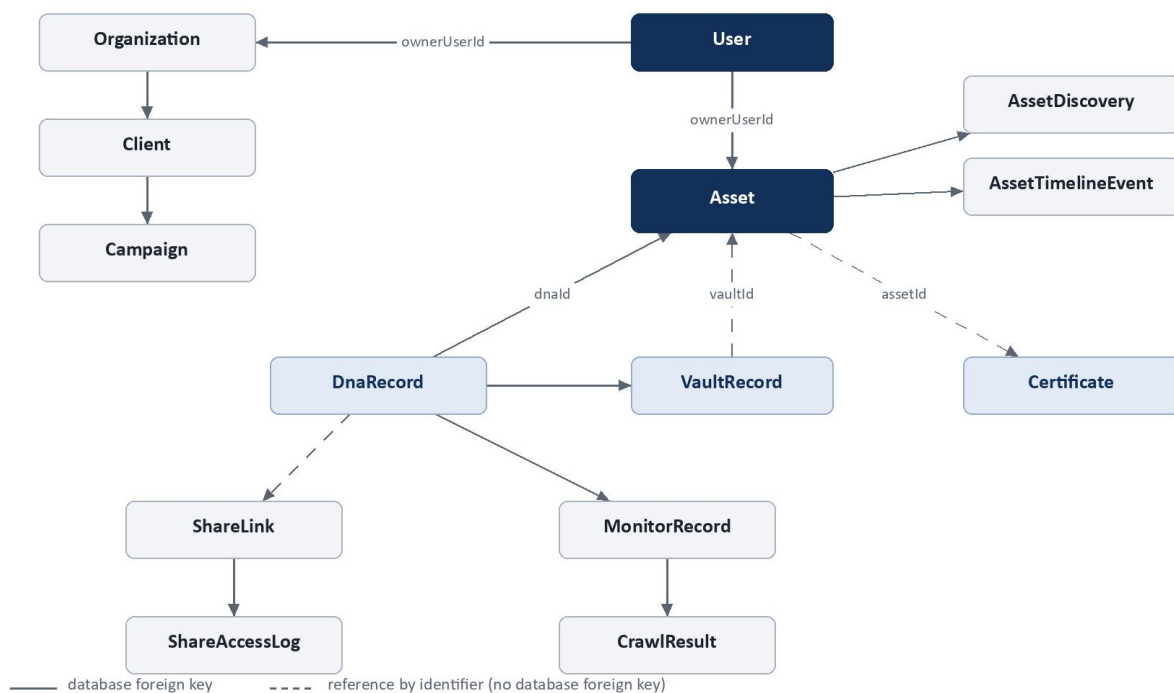


Figure 8. HUB core ER diagram

B. Core entity relationship table

Entity	Purpose	Primary key	Important relationships	Used by
User	Account and identity	id	Owns Asset, Organization, ShareLink, Certificate (ownerUserId)	Every module
Asset	Canonical asset identity	id	ownerUser; dnaRecord (FK dnaId); timeline, discoveries, versions; vaultId and certificateId by value	Vault, sharing, Exchange, portfolio

Entity	Purpose	Primary key	Important relationships	Used by
DnaRecord	Pinit DNA of a file	id	ownerUser; layer records; VaultRecord and MonitorRecord (FK)	Protection, verification, monitoring
VaultRecord	Encrypted stored copy	id	dnaRecord (FK dnaRecordId, unique, cascade)	Vault, sharing
Certificate	Signed proof	id (certificateId unique)	ownerUser (FK); dnaRecordId, vaultId, assetId by value	Certificates, credentials
Credential	Credential projection of a certificate	id	user (FK); unique (sourceType, sourceId)	Credentials screen
ShareLink	A secure link and its rules	id (token unique)	ownerUser, shareRecipient, parentLink (FK); vaultId, dnaRecordId, assetId by value	Sharing, business handover
ShareAccessLog	One access to a link	id	shareLink (FK, cascade)	Link intelligence
MonitorRecord	A monitoring profile	id	dnaRecord, ownerUser (FK)	Monitoring
CrawlResult	A scanned result	id	monitorRecord (FK, cascade)	Monitoring
AssetDiscovery	A copy found online	id	asset (FK, cascade)	Monitoring
AssetTimelineEvent	Recorded event on an asset	id	asset (FK, cascade)	Asset history
Organization	A business	id	ownerUser (FK); Client, Workspace, OrganizationMember	Business workspace
Client / Campaign	Client and project	id	Client to organization; Campaign to organization and client (FK)	Business workspace
CampaignMember / CampaignMemberAsset	People and their assets	id	CampaignMember to campaign; CampaignMemberAsset to member (FK)	Business workspace
Portfolio	A creator portfolio	id (userId unique)	user (FK); projects, media (PortfolioProjectMedia.assetId and vaultId by value)	Portfolio
PlatformEvent	Event stream	id	ownerUser (FK); entityType and entityId by value	Notifications, audit
Subscription / Plan	Plan and entitlement state	id	Subscription to user and plan	Feature gating

C. Data lifecycle



Figure 9. Order in which records are created for a protected asset

The order differs from the ownership view in Section 8 because the DNA record is generated first; the Asset row is created when the file is stored in the Vault.

10. Backend Architecture

Item	Implementation
Framework	Node.js with Express 4, TypeScript (src/server.ts, src/app.ts)
Structure	src/api/{routes, controllers, middleware}; src/services/*; src/lib; src/config; prisma/schema.prisma
Authentication	JWT access tokens (Bearer) verified by requireAuth; HttpOnly refresh cookie (pinit_rt); bcrypt for password hashes
Authorization	Ownership middleware (requireVaultOwnership, requireDnaOwnership, requireCertificateOwnership, requireShareLinkOwnership) and tenant scope helpers
Feature gating	requireFeature middleware backed by plan entitlements
Database access	Prisma client; a small number of read-only reporting queries use raw SQL
File storage	Supabase Storage (bucket vault-files); local disk in development
Encryption	AES-256-GCM; key = HKDF-SHA256(master secret, salt = vault id); random 96-bit IV per file
External services	Python AI service (FastAPI, port 8001), Razorpay, Supabase
Events and audit	Platform event engine with notification, timeline and audit subscribers; forensic provenance events; organization audit log
Security middleware	helmet (content-security-policy disabled), CORS, a global API rate limit and a stricter limit on authentication routes, input validation

Service domains implemented

Domain	Location	Responsibility
DNA	services/layers, dna, dna.orchestrator	Layer computation and DNA records
Vault	services/vault	Encryption, storage, retrieval, protected download
Assets	services/assets	Canonical asset identity and linkage
Certificates	services/certificates, services/credentials	Issue, verify, revoke, credential projection
Sharing	services/share	Link rules, access checks, risk engine, review and approval
Monitoring	services/crawler	Crawler engine, connectors, scoring
Organization	services/organization	Teams, roles, workspaces
Platform events	services/platform-events	Event engine, notifications, timeline
Subscription	services/subscription	Plans, entitlements, feature checks
Exchange bridge	services/exchange	Hub to Exchange calls and single sign-on

11. API Architecture

The tables below were generated from the route files (Hub: src/api/routes; Exchange: exchange/server/routes), so every listed endpoint exists in code. **Route-level guard** shows the middleware declared on the route. Exchange resolves the buyer or seller identity from its session token inside many handlers, so “None at route level” does not mean the endpoint is unauthenticated. Request and response bodies are defined in the corresponding controllers and are not reproduced here.

Endpoints that belong to modules outside the completed scope, and endpoints whose feature has no evidence of use, are omitted (for example organization API keys, webhooks and integrations; business investigations, evidence and reports; spatial authentication; Exchange refunds, coupons and reviews).

11.1 Pinit HUB API (base path /api/v1)

Authentication (17)

Database entities: User, RefreshToken, FaceTemplate, BiometricIdentity, WebAuthnCredential, UserSession, LoginHistory, SecurityEvent

Method	Endpoint	Resource / function	Route-level guard
POST	/auth/create	Create account	None at route level
POST	/auth/login	Auth: login	None at route level
POST	/auth/refresh	Refresh	None at route level
POST	/auth/logout	Logout	None at route level
GET	/auth/me	Auth: me	Bearer JWT required
POST	/auth/account-type	Set account type	Bearer JWT required
POST	/auth/business-setup	Setup business workspace	Bearer JWT required
GET	/auth/business-setup/status	Business setup: status	Bearer JWT required
POST	/auth/face/challenge	Face: challenge	None at route level
POST	/auth/face/register	Face: register	None at route level
POST	/auth/face/login	Face: login	None at route level
POST	/auth/face/identify	Face: identify	None at route level
GET	/auth/face/status	Face: status	Bearer JWT required
POST	/auth/passkey/register/start	Register: start	None at route level
POST	/auth/passkey/register/finish	Register: finish	None at route level
POST	/auth/passkey/login/start	Login: start	None at route level
POST	/auth/passkey/login/finish	Login: finish	None at route level

Users and Profile (12)

Database entities: User, UserSession, Notification preferences

Method	Endpoint	Resource / function	Route-level guard
GET	/profile/avatar/:shortId	Get public avatar	None at route level
POST	/profile/avatar	Upload profile avatar	Bearer JWT required

Method	Endpoint	Resource / function	Route-level guard
DELETE	/profile/avatar	Delete profile avatar	Bearer JWT required
GET	/profile/	Get profile	Bearer JWT required
PUT	/profile/	Update profile	Bearer JWT required
PUT	/profile/notifications	Update notification prefs	Bearer JWT required
PUT	/profile/password	Profile: password	Bearer JWT required
GET	/profile/stats	Get profile stats	Bearer JWT required
GET	/profile/activity	Get activity timeline	Bearer JWT required
GET	/profile/sessions	Get sessions	Bearer JWT required
DELETE	/profile/session/:id	Revoke session	Bearer JWT required
DELETE	/profile/sessions	Revoke all sessions	Bearer JWT required

Assets (6)

Database entities: Asset, AssetTimelineEvent, AssetPlatformLink

Method	Endpoint	Resource / function	Route-level guard
POST	/assets/protect	Protect asset	Bearer JWT required
GET	/assets/stats	Get asset stats	Bearer JWT required
GET	/assets	List assets	Bearer JWT required
GET	/assets/:id/graph	Get asset graph	Bearer JWT required
GET	/assets/:id	Get asset	Bearer JWT required
PATCH	/assets/:id/status	Transition asset	Bearer JWT required

DNA (8)

Database entities: DnaRecord and the layer records, VerificationLog

Method	Endpoint	Resource / function	Route-level guard
GET	/dna/	List dna records	Bearer JWT required
GET	/dna/supported-types	Get supported types	None at route level
GET	/dna/duplicate-attempts	Get duplicate attempts	Bearer JWT required
POST	/dna/generate	Generate dna	Bearer JWT required
POST	/dna/compare	Compare dna	Bearer JWT required
POST	/dna/auto-compare	Dna: auto compare	Bearer JWT required
POST	/dna/:id/verify	Verify dna	Bearer JWT required
GET	/dna/:id	Get dna record	Bearer JWT required

Vault (15)

Database entities: VaultRecord, Asset, TrackedExportPackage

Method	Endpoint	Resource / function	Route-level guard
--------	----------	---------------------	-------------------

Method	Endpoint	Resource / function	Route-level guard
GET	/vault/	List vault records	Bearer JWT required
GET	/vault/integrity-check	Vault: integrity check	Bearer JWT required
GET	/vault/protected-shares	List protected file shares	Bearer JWT required
POST	/vault/store	Store in vault	Bearer JWT required
GET	/vault/:id	Get vault record	Bearer JWT required
PATCH	/vault/:id/rename	Rename vault record	Bearer JWT required
DELETE	/vault/:id	Delete vault record	Bearer JWT required
GET	/vault/:id/preview	Preview vault file	Bearer JWT required
POST	/vault/:id/retrieve	Retrieve from vault	Bearer JWT required
POST	/vault/:id/protected-download/prepare	Prepare protected download	Bearer JWT required
POST	/vault/:id/protected-download	Protected download from vault	Bearer JWT required
GET	/vault/:id/tracking	Get vault tracking	Bearer JWT required
POST	/vault/:id/tep/:tepCode/revoke	Revoke vault tep	Bearer JWT required
GET	/tep/manifests	List tep manifests	Bearer JWT required
GET	/tep/:tepCode	Get tep manifest	Bearer JWT required

Certificates and Credentials (7)

Database entities: Certificate, Credential

Method	Endpoint	Resource / function	Route-level guard
POST	/certificates/	Issue certificate	Bearer JWT required
GET	/certificates/verify/:certificateId	Verify certificate	None at route level
POST	/certificates/revoke/:certificateId	Revoke certificate	Bearer JWT required
GET	/certificates/	List certificates	Bearer JWT required
GET	/certificates/dna/:dnaRecordId	List certificates by dna	Bearer JWT required
GET	/credentials/me	List my credentials handler	Bearer JWT required
GET	/credentials/:credentialId	Get my credential handler	Bearer JWT required

Sharing (38)

Database entities: ShareLink, ShareRecipient, ShareAccessLog, LinkForwardEvent, BlockedShareViewer

Method	Endpoint	Resource / function	Route-level guard
POST	/share/	Create share link	Bearer JWT required
POST	/share/file	Create file share	Bearer JWT required
GET	/share/	List share links	Bearer JWT required
GET	/share/vault/:vaultId	Get vault share links	Bearer JWT required
GET	/share/timeline/:dnald	Get share timeline	Bearer JWT required
GET	/share/analytics/geo	Get geo analytics	Bearer JWT required

Method	Endpoint	Resource / function	Route-level guard
GET	/share/analytics/global	Get global share stats	Bearer JWT required
GET	/share/analytics/live-map	Get live tracking map	Bearer JWT required
GET	/share/sessions/live	Get live sessions	Bearer JWT required
GET	/share/messages	List share viewer messages	Bearer JWT required
POST	/share/messages/:id/reply	Reply share viewer message	Bearer JWT required
GET	/share/:token	Get share link info	None at route level
POST	/share/:token/access	Record access	None at route level
POST	/share/:token/share-further	Share further	None at route level
POST	/share/:token/verify-otp	Verify share otp	None at route level
GET	/share/:token/file	Share: file	None at route level
GET	/share/:token/preview.png	Preview image	None at route level
GET	/share/:token/masked-text	Get masked text	None at route level
POST	/share/:token/messages	Post share viewer message	None at route level
GET	/share/:token/messages/mine	Get my share viewer messages	None at route level
GET	/share/:token/review	Get share review	None at route level
GET	/share/:token/review/comments	Get share review comments	None at route level
POST	/share/:token/review/comments	Post share review comment	None at route level
GET	/share/:token/review/decisions	Get share review decisions	None at route level
POST	/share/:token/review/decision	Post share review decision	None at route level
GET	/share/:token/messages/stream	Stream share messages	None at route level
GET	/share/handover/:token	Get handover view	None at route level
GET	/share/:token/logs	Get share link logs	Bearer JWT required
GET	/share/:token/export	Export share logs csv	Bearer JWT required
DELETE	/share/:token	Revoke share link	Bearer JWT required
POST	/share/:token/block-viewer	Block share viewer	Bearer JWT required
DELETE	/share/:token/block-viewer/:blockId	Share: block viewer (by id)	Bearer JWT required
POST	/share/:token/force-logout	Force logout link	Bearer JWT required
GET	/share/:token/tree	Get link tree	Bearer JWT required
GET	/recipients/	List recipients	Bearer JWT required
POST	/recipients/	Create recipient	Bearer JWT required
GET	/recipients/:id	Get recipient	Bearer JWT required
DELETE	/recipients/:id	Delete recipient	Bearer JWT required

Monitoring (15)

Database entities: MonitorRecord, MonitoringRun, CrawlResult, CrawlerMatch, AssetDiscovery

Method	Endpoint	Resource / function	Route-level guard
--------	----------	---------------------	-------------------

Method	Endpoint	Resource / function	Route-level guard
GET	/monitor/stats	Get monitoring stats	Bearer JWT required
GET	/monitor/engine/stats	Get engine stats	Bearer JWT required
POST	/monitor/enroll-all	Enroll all	Bearer JWT required
GET	/monitor/	List monitors	Bearer JWT required
POST	/monitor/enroll/:dnaRecordId	Enroll monitor	Bearer JWT required
GET	/monitor/alerts	Get alerts	Bearer JWT required
POST	/monitor/alerts/:id/dismiss	Dismiss alert	Bearer JWT required
POST	/monitor/alerts/:id/confirm	Confirm alert	Bearer JWT required
POST	/monitor/:id/check	Run check now	Bearer JWT required
GET	/monitor/:id/runs	Get monitor runs	Bearer JWT required
PATCH	/monitor/:id/scan-type	Update scan type	Bearer JWT required
PATCH	/monitor/:id/watch-urls	Update watch urls	Bearer JWT required
POST	/monitor/:id/pause	Pause monitor	Bearer JWT required
POST	/monitor/:id/resume	Resume monitor	Bearer JWT required
DELETE	/monitor/:id	Stop monitor	Bearer JWT required

Notifications (7)

Database entities: Notification, PlatformEvent

Method	Endpoint	Resource / function	Route-level guard
GET	/notifications/stream	Stream notifications	Role / ownership guard
GET	/notifications/	Get notifications	Bearer JWT required
PUT	/notifications/read-all	Mark all read	Bearer JWT required
PUT	/notifications/clear-inbox	Clear inbox	Bearer JWT required
PUT	/notifications/:id/read	Mark read	Bearer JWT required
PUT	/notifications/:id/archive	Archive notification	Bearer JWT required
DELETE	/notifications/:id	Delete notification	Bearer JWT required

Business: Organization, Team, Workspaces (22)

Database entities: Organization, OrganizationMember, OrganizationInvite, Department, Workspace, OrganizationAuditLog

Method	Endpoint	Resource / function	Route-level guard
GET	/organization/me	Get mine	Bearer JWT required
POST	/organization/setup	Organization: setup	Bearer JWT required
PATCH	/organization/profile	Update profile	Bearer JWT required
POST	/organization/logo	Upload logo	Bearer JWT required
POST	/organization/welcome/skip	Skip welcome	Bearer JWT required
GET	/organization/team	Get team	Bearer JWT required

Method	Endpoint	Resource / function	Route-level guard
GET	/organization/team/members	List members	Bearer JWT required
GET	/organization/team/invites	List invites	Bearer JWT required
POST	/organization/team/invite	Invite member	Bearer JWT required
GET	/organization/team/invites/preview/:token	Preview invite	Bearer JWT required
POST	/organization/team/accept	Accept invite	Bearer JWT required
GET	/organization/team/lookup-pinit-id	Lookup pinit id	Bearer JWT required
DELETE	/organization/team/invites/:id	Revoke invite	Bearer JWT required
PATCH	/organization/team/members/:id/role	Update member role	Bearer JWT required
DELETE	/organization/team/members/:id	Remove member	Bearer JWT required
GET	/organization/workspaces	List workspaces	Bearer JWT required
POST	/organization/workspaces	Create workspace	Bearer JWT required
GET	/organization/departments	List departments	Bearer JWT required
POST	/organization/departments	Create department	Bearer JWT required
PATCH	/organization/departments/:id	Update department	Bearer JWT required
DELETE	/organization/departments/:id	Delete department	Bearer JWT required
GET	/organization/audit-logs	List audit logs	Bearer JWT required

Business: Clients, Campaigns, People, Review (51)

Database entities: Client, Campaign, CampaignMember, CampaignMemberAsset, AssetVersion, VersionApproval, ReviewComment, CampaignMessage, CampaignHandover

Method	Endpoint	Resource / function	Route-level guard
GET	/business/overview	Get overview	Bearer JWT required
GET	/business/clients	List clients	Bearer JWT required
POST	/business/clients	Create client	Bearer JWT required
GET	/business/clients/:clientId	Get client	Bearer JWT required
PATCH	/business/clients/:clientId	Update client	Bearer JWT required
DELETE	/business/clients/:clientId	Delete client	Bearer JWT required
GET	/business/clients/:clientId/campaigns	List campaigns	Bearer JWT required
POST	/business/clients/:clientId/campaigns	Create campaign	Bearer JWT required
GET	/business/campaigns/:campaignId	Get campaign	Bearer JWT required
PATCH	/business/campaigns/:campaignId	Update campaign	Bearer JWT required
DELETE	/business/campaigns/:campaignId	Delete campaign	Bearer JWT required
GET	/business/campaigns/:campaignId/members	List campaign members	Bearer JWT required
POST	/business/campaigns/:campaignId/members	Campaigns: members	Bearer JWT required
DELETE	/business/campaigns/:campaignId/members/:memberId	Remove campaign member	Bearer JWT required
GET	/business/campaigns/:campaignId/assets	List campaign assets	Bearer JWT required

Method	Endpoint	Resource / function	Route-level guard
POST	/business/campaigns/:campaignId/assets	Campaigns: assets	Bearer JWT required
GET	/business/campaigns/:campaignId/activity	List campaign activity	Bearer JWT required
GET	/business/assets/:assetId/versions	List asset versions	Bearer JWT required
POST	/business/assets/:assetId/versions	Create asset version	Bearer JWT required
GET	/business/versions/:versionId	Get asset version	Bearer JWT required
GET	/business/versions/:versionId/file	Versions: file	Bearer JWT required
PATCH	/business/versions/:versionId/review-status	Set version review status	Bearer JWT required
GET	/business/versions/:versionId/comments	List version comments	Bearer JWT required
POST	/business/versions/:versionId/comments	Create version comment	Bearer JWT required
PATCH	/business/comments/:commentId/status	Set comment status	Bearer JWT required
GET	/business/campaigns/:campaignId/change-requests	List campaign change requests	Bearer JWT required
POST	/business/versions/:versionId/decision	Decide version	Bearer JWT required
GET	/business/versions/:versionId/approvals	List version approvals	Bearer JWT required
GET	/business/campaigns/:campaignId/approvals	List campaign approvals	Bearer JWT required
GET	/business/share-eligibility/:vaultId	Get share review eligibility	Bearer JWT required
GET	/business/campaigns/:campaignId/messages	List campaign messages	Bearer JWT required
POST	/business/campaigns/:campaignId/messages	Send campaign message	Bearer JWT required
POST	/business/campaigns/:campaignId/messages/read	Mark campaign messages read	Bearer JWT required
GET	/business/campaigns/:campaignId/messages/stream	Stream campaign messages	Role / ownership guard
GET	/business/messages/unread	Get campaign unread	Bearer JWT required
GET	/business/campaigns/:campaignId/people	List campaign people	Bearer JWT required
POST	/business/campaigns/:campaignId/people/:memberId/access	Grant campaign access	Bearer JWT required
PATCH	/business/campaigns/:campaignId/people/:memberId/access	Update campaign access	Bearer JWT required
DELETE	/business/campaigns/:campaignId/people/:memberId/access	Revoke campaign access	Bearer JWT required
GET	/business/campaigns/:campaignId/people/:memberId/links	List campaign access links	Bearer JWT required
GET	/business/campaigns/:campaignId/rights	List campaign rights	Bearer JWT required
GET	/business/campaigns/:campaignId/handover/candidates	List handover candidates	Bearer JWT required
GET	/business/campaigns/:campaignId/handovers	List handovers	Bearer JWT required
POST	/business/campaigns/:campaignId/handovers	Create handover	Bearer JWT required
POST	/business/campaigns/:campaignId/handovers/:handoverId/send	Send handover	Bearer JWT required
DELETE	/business/campaigns/:campaignId/handovers/:handoverId	Revoke handover	Bearer JWT required
GET	/business/clients/:clientId/deliveries	Clients: deliveries	Bearer JWT required
GET	/business/clients/:clientId/rights	Clients: rights	Bearer JWT required

Method	Endpoint	Resource / function	Route-level guard
GET	/business/clients/:clientId/activity	Clients: activity	Bearer JWT required
GET	/business/clients/:clientId/assets	Clients: assets	Bearer JWT required
GET	/business/clients/:clientId/people	Clients: people	Bearer JWT required

Portfolio (7)

Database entities: Portfolio, PortfolioProfile, PortfolioProject, PortfolioProjectMedia

Method	Endpoint	Resource / function	Route-level guard
GET	/portfolio/me	Get my portfolio	Bearer JWT required
PUT	/portfolio/me	Portfolio: me	Bearer JWT required
POST	/portfolio/me/publish	Publish my portfolio	Bearer JWT required
POST	/portfolio/me/unpublish	Unpublish my portfolio	Bearer JWT required
GET	/portfolio/me/preview	Preview my portfolio	Bearer JWT required
GET	/portfolio/me/contains/:vaultId	Me: contains (by id)	Bearer JWT required
GET	/portfolio/public/:slug	Get public portfolio	None at route level

Subscription and Entitlements (4)

Database entities: Plan, Subscription, FeatureEntitlement, BillingHistory, UsageRecord

Method	Endpoint	Resource / function	Route-level guard
GET	/subscription/me	Get my subscription	Bearer JWT required
GET	/subscription/plans	List subscription plans	Bearer JWT required
GET	/subscription/billing/config	Get billing config	Bearer JWT required
GET	/subscription/billing/history	Get billing history	Bearer JWT required

11.2 Pinit Exchange API (base path /api)

Exchange Authentication and Session (9)

Database entities: users

Method	Endpoint	Resource / function	Route-level guard
GET	/auth/me	Auth: me	None at route level
POST	/auth/signup	Auth: signup	None at route level
POST	/auth/login	Auth: login	None at route level
POST	/auth/hub-sso	Auth: hub sso	None at route level
POST	/auth/profile	Auth: profile	None at route level
POST	/auth/become-creator	Auth: become creator	Session required
POST	/auth/enable-buyer	Auth: enable buyer	Role / ownership guard
POST	/auth/onboard-seller	Auth: onboard seller	Role / ownership guard
POST	/auth/logout	Auth: logout	None at route level

Hub Bridge (Exchange side) (11)

Database entities: hub_assets, listings, hub_bridge_events

Method	Endpoint	Resource / function	Route-level guard
GET	/hub/assets	Hub: assets	Role / ownership guard
POST	/hub/protect-upload	Hub: protect upload	Role / ownership guard
GET	/hub/preview/:assetId	Hub: preview (by id)	None at route level
GET	/hub/monitoring-summaries	Hub: monitoring summaries	None at route level
POST	/hub/protect	Hub: protect	None at route level
GET	/hub/portfolios	Hub: portfolios	Role / ownership guard
GET	/hub/portfolio	Hub: portfolio	Role / ownership guard
PUT	/hub/portfolio	Hub: portfolio	Role / ownership guard
GET	/hub/seller-listings	Hub: seller listings	Role / ownership guard
GET	/hub/seller-desk	Hub: seller desk	Role / ownership guard
GET	/hub/buyer-desk	Hub: buyer desk	Role / ownership guard

Listings (6)

Database entities: listings, hub_assets

Method	Endpoint	Resource / function	Route-level guard
GET	/listings/	Listings	None at route level
GET	/listings/:id	Listings (by id)	None at route level
POST	/listings/from-hub	Listings: from hub	Role / ownership guard
POST	/listings/	Listings	Role / ownership guard
POST	/listings/:id/status	Listings: status	Role / ownership guard
PATCH	/listings/:id	Listings (by id)	Role / ownership guard

Orders, Payments and Licences (10)

Database entities: orders_sealed, payment_intents, asset_commerce_locks, seller_earnings, hub_bridge_events

Method	Endpoint	Resource / function	Route-level guard
GET	/orders/billing/config	Billing: config	None at route level
POST	/orders/create-payment	Orders: create payment	Role / ownership guard
POST	/orders/verify-payment	Orders: verify payment	None at route level
POST	/orders/checkout	Orders: checkout	Role / ownership guard
GET	/orders/my-licenses	Orders: my licenses	Session required
GET	/orders/certificate/:seal_id	Orders: certificate (by id)	Session required
POST	/orders/download/authorize	Download: authorize	Session required
GET	/orders/my-orders	Orders: my orders	Session required

Method	Endpoint	Resource / function	Route-level guard
GET	/orders/invoice/:sealId	Orders: invoice (by id)	Session required
POST	/orders/bridge-events/retry	Bridge events: retry	None at route level

Cart, Wishlist and Follows (14)

Database entities: cart_items, wishlist, creator_follows, orders_sealed

Method	Endpoint	Resource / function	Route-level guard
GET	/commerce/cart	Commerce: cart	None at route level
POST	/commerce/cart	Commerce: cart	None at route level
DELETE	/commerce/cart/:id	Commerce: cart (by id)	None at route level
POST	/commerce/cart/create-payment	Cart: create payment	Role / ownership guard
POST	/commerce/cart/checkout	Cart: checkout	Role / ownership guard
GET	/commerce/wishlist	Commerce: wishlist	None at route level
POST	/commerce/wishlist	Commerce: wishlist	None at route level
DELETE	/commerce/wishlist/:listingId	Commerce: wishlist (by id)	None at route level
GET	/commerce/follows	Commerce: follows	None at route level
POST	/commerce/follows	Commerce: follows	None at route level
DELETE	/commerce/follows/:creatorId	Commerce: follows (by id)	None at route level
POST	/commerce/claim-guest	Commerce: claim guest	Session required
POST	/commerce/purchases/:sealId/share	Purchases: share	Session required
POST	/commerce/purchases/:sealId/share/copied	Share: copied	Session required

Creators (6)

Database entities: users, listings, orders_sealed

Method	Endpoint	Resource / function	Route-level guard
GET	/creator/desk	Creator: desk	Role / ownership guard
GET	/creator/listings	Creator: listings	Role / ownership guard
GET	/creator/directory	Creator: directory	None at route level
GET	/creator/profile/:pinitId	Creator: profile (by id)	None at route level
GET	/creator/assets/:assetId/activity	Assets: activity	Role / ownership guard
GET	/creator/collectors	Creator: collectors	None at route level

Opportunities (briefs and proposals) (18)

Database entities: requirements and the proposal records

Method	Endpoint	Resource / function	Route-level guard
GET	/opportunities/briefs	Opportunities: briefs	Role / ownership guard
GET	/opportunities/briefs/:reqId	Opportunities: briefs (by id)	Role / ownership guard

Method	Endpoint	Resource / function	Route-level guard
POST	/opportunities/briefs/:reqId/propose	Briefs: propose	Role / ownership guard
POST	/opportunities/briefs/:reqId/withdraw	Briefs: withdraw	Role / ownership guard
GET	/opportunities/my-briefs	Opportunities: my briefs	Role / ownership guard
GET	/opportunities/briefs/:reqId/proposals	Briefs: proposals	Role / ownership guard
POST	/opportunities/briefs/:reqId/award	Briefs: award	Role / ownership guard
POST	/opportunities/briefs/:reqId/invite	Briefs: invite	Role / ownership guard
POST	/opportunities/briefs/:reqId/questions	Briefs: questions	None at route level
POST	/opportunities/briefs/:reqId/team	Briefs: team	Role / ownership guard
POST	/opportunities/teams/:teamId/accept	Teams: accept	Role / ownership guard
GET	/opportunities/creators	Opportunities: creators	Role / ownership guard
GET	/opportunities/collabs	Opportunities: collabs	Role / ownership guard
POST	/opportunities/collabs	Opportunities: collabs	Role / ownership guard
POST	/opportunities/collabs/:collabId/close	Collabs: close	Role / ownership guard
POST	/opportunities/asks	Opportunities: asks	Role / ownership guard
POST	/opportunities/asks/:askId/respond	Asks: respond	Role / ownership guard
GET	/opportunities/activity	Opportunities: activity	Role / ownership guard

Seller Onboarding (4)

Database entities: seller_onboarding_intents, seller_payment_methods

Method	Endpoint	Resource / function	Route-level guard
GET	/seller/onboarding/status	Onboarding: status	Session required
POST	/seller/onboarding/payment-method	Onboarding: payment method	Session required
POST	/seller/onboarding/payment-method/verify	Payment method: verify	Session required
POST	/webhooks/razorpay	Webhooks: razorpay	None at route level

12. Security and Authentication

Area	Implemented behaviour	Evidence
Identity creation	Account creation issues a unique PINIT ID. Biometric embeddings are never returned in API responses or written to logs.	[T] pinit-id-uniqueness, no-embedding-in-response, no-embedding-in-logs
Authentication	Password (bcrypt), face verification with presentation-attack (liveness) checks, passkeys (WebAuthn).	[T] face-liveness-pad, face-match-gate, webauthn-passkey
Sessions	Short-lived access JWT sent as Bearer; refresh token in an HttpOnly cookie (SameSite none over HTTPS, lax otherwise); sessions can be listed and revoked.	[T] auth-cookies, jwt-subject-and-isolation
Authorization	Authentication factors only mint a JWT; every protected API still requires JWT subject equal to the resource owner. A biometric mis-identification cannot grant ownership.	[T] authz-boundary, tenant-isolation, jwt-subject-and-isolation
Asset ownership	Ownership middleware on DNA, vault, certificate and share routes; owner-scoped queries.	[T] owner-binding
Protected storage	AES-256-GCM with a per-record derived key; authentication tag detects tampering on decrypt.	[T] vault-blob-integrity
Certificate verification	HMAC-SHA256 signature checked with a constant-time comparison; revoked and expired certificates never verify.	[T] credentials-phase1
Share access control	Expiry, view and download limits, one-time use, OTP, country, device and IP rules, viewer blocking, forced logout, revocation.	[T] share suites; [O] 195 access-log rows
Audit and activity	Security events, login history, share access logs, platform events, forensic provenance events, organization audit log.	[O] events recorded

Not represented as production-ready

Passkeys are covered by tests but no production enrolment is recorded. Helmet runs with its content-security-policy disabled. This document does not assert a formal security audit or penetration test, because none is present in the repository.

13. Sharing and Access Control

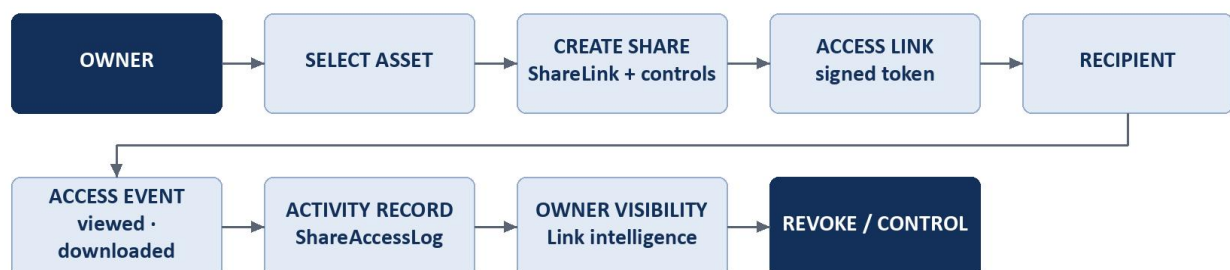


Figure 12. Secure sharing flow

Stage	Implemented
Share creation	Owner selects a protected asset and creates a ShareLink (POST /share). A signed token is issued.
Recipient and access model	Optional recipient e-mail (ShareRecipient); recipient name capture; OTP verification when required.

Stage	Implemented
Access rules enforced	Expiry date, maximum views, maximum downloads, one-time use, allowed countries, allowed device types, allowed IP prefixes, download and print permission, privacy masking options.
View and download	Public viewer page (ShareViewerPage); masked output where privacy masking is configured.
Tracking	Every access is written to ShareAccessLog with device, browser, operating system and location, and a risk assessment.
Activity	PlatformEvents notify the owner; link intelligence pages show accesses, a live map and the forwarding chain (parent and child links).
Onward sharing	A recipient can share further where allowed; the chain is recorded (parentLinkId, depth, forwardedByLabel).
Revoke and control	Owner revokes a link, blocks a viewer, forces a logout, exports the log to CSV.

Relationship to Exchange

A licensed purchase in Exchange is delivered through a share link created in HUB by the bridge (POST /exchange/share/create), so buyers receive licensed access under the same access controls.

14. Certificate and Verification System

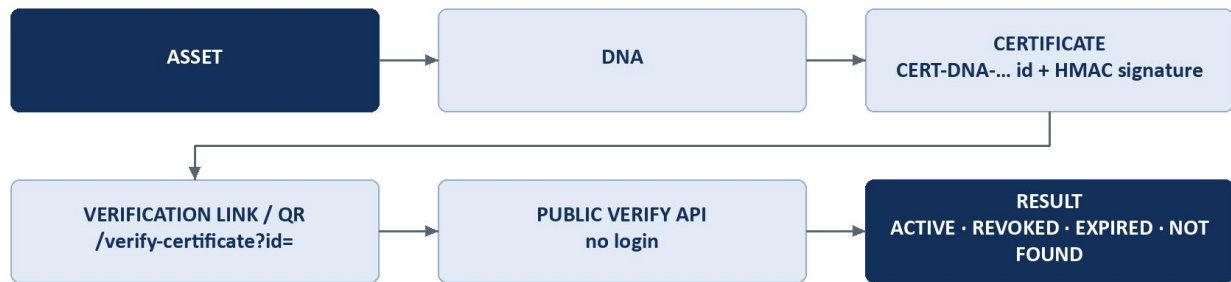


Figure 13. Certificate verification flow

Concept	Role
DNA	The asset identity: what the file is
Certificate	The verifiable proof that the identity was recorded, by whom and when
Vault	The protected file storage
Activity	The lifecycle history of what happened afterwards

Certificate creation	POST /certificates issues a certificate for a DNA record and vault record owned by the caller. If an ACTIVE certificate already exists for the pair, that certificate is returned instead of creating a second.
Certificate ID	CERT-DNA-<UUID>, unique.
Relationships	Certificate.dnaRecordId and vaultId are signed into the certificate; assetId links the certificate to the asset; ownerUserId is the owner.
Signature	HMAC-SHA256 over a canonical string of the certificate id, DNA id, vault id and issue time, keyed from the vault master secret.
Verification	GET /certificates/verify/:certificateId is public. It returns ACTIVE with signatureValid, REVOKED with the date and reason, EXPIRED, or NOT_FOUND. An invalid signature is reported as a possible forgery.
QR / link	The certificate document carries a QR code to the verification page (/verify-certificate?id=...) of the HUB deployment that rendered it. Verification needs no login.
Revocation	POST /certificates/revoke/:certificateId, owner only, with a mandatory reason; sets status REVOKED, the time and the reason.
Expiry	An optional expiry is supported; a scheduler and the verify call move an expired certificate to EXPIRED.
Immutability	There is no update or edit endpoint. A certificate can only be issued, revoked or expire; its record and signed fields are not altered afterwards. Revoked certificates remain stored, so the original stays identifiable.
Issue time	Stored exactly (millisecond timestamp) and part of the signed string; the document shows the date.

Known limitations of the committed baseline

The public verification response returns the full certificate record, including its internal DNA record id, vault id and signature value. A restricted public view that omits them is implemented in the working tree but is not committed (see final section).

The certificate's signed string does not include the asset id or the file hash; an asset link added after issue is not covered by the signature. A version that seals both is implemented in the working tree but is not committed.

The certificate document does not yet carry the evidence-record disclaimer. Pinit certificates record protection evidence; they are not a government copyright registration and are not a legal determination of ownership.

15. Monitoring and Intelligence

In the completed scope, intelligence means monitoring matches and the link intelligence described in Section 13 (who opened a link, from where, and where it was forwarded). Investigation is not included.

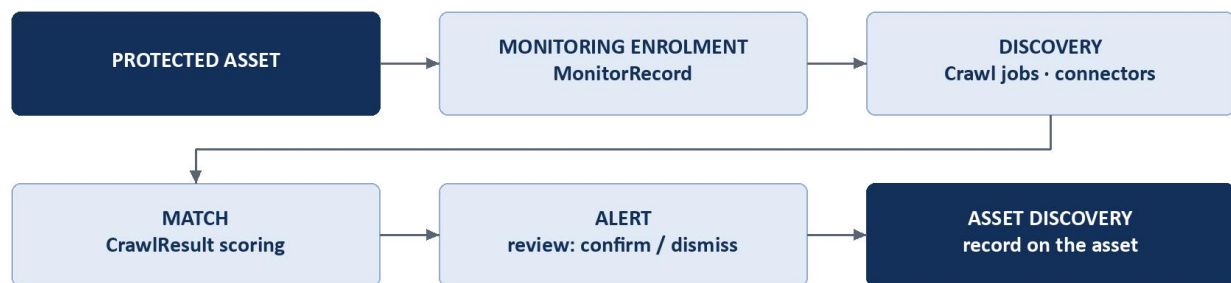


Figure 14. Monitoring flow

Stage	Implemented
Enrolment	An asset is enrolled (POST /monitor/enroll/:dnaRecordId or enroll-all), creating a MonitorRecord with scan type and optional watch URLs.
Supported sources	The crawler engine has connectors for YouTube, Reddit, GitHub, Telegram and general websites , plus filename-search and image-search providers. Browser-automation crawling is disabled by default.
Execution	Scans run through a queue and scheduler; each run is recorded (MonitoringRun) and failures are recorded (MonitoringFailure).
Discovered content	Results are stored as CrawlResult rows.
Matching	Results are scored against the asset's DNA; matches are stored as CrawlerMatch and, where confirmed, AssetDiscovery.
Alerts	Alerts are listed, and the owner confirms or dismisses each (POST /monitor/alerts/:id/confirm and /dismiss); notifications are raised.

Boundaries

Monitoring covers the listed sources only. It is not universal internet monitoring.

Investigation and evidence reporting after an alert is outside the completed scope (final section). In production data the monitoring engine has executed at scale (20,527 results) but no confirmed discovery is recorded for the largest account.

16. Business Workspace

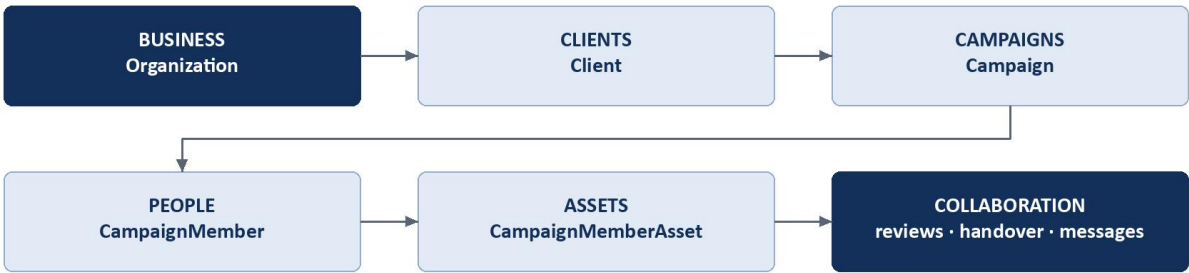


Figure 15. Business Workspace flow

Example. A creative agency creates a business workspace, adds its client “Northwind”, opens a campaign “Spring Launch” under that client, invites team members, attaches the protected assets for the launch, sends versions for review, records the client’s approval or requested changes, and finally hands the approved assets over through a secure link.

Level	Implemented
Business (Organization)	Profile and logo, team, invites, roles, departments, workspaces, audit log.
Clients	Create, edit and delete clients; client workspace with campaigns, deliveries, rights and activity.
Campaigns	Create campaigns under a client; campaign workspace with members, assets, activity and messages.
People	Campaign members with per-person access to selected assets and a view of the links each person holds.
Assets	Protected assets attached to a campaign; asset versions with review status.
Collaboration	Review comments, approvals and change requests on versions, campaign messages, handovers to the client.

	Personal Workspace	Business Workspace
Owner	One creator	An organization with members
Structure	Assets, sharing, monitoring, portfolio	Clients, campaigns, people, versions, handovers on top of assets
Access	The owner	Owner and members by role; per-person asset access within a campaign

Roles and permissions are those implemented in the organization and campaign services (member roles, campaign member access records). Intelligence, investigation and report views inside the business workspace are outside the completed scope.

17. Pinit Exchange: Product Overview

Pinit Exchange is the marketplace layer for discovering, listing, buying, selling and licensing digital creative work. **HUB protects and manages; Exchange markets and transacts.**

Item	Implementation
Frontend	React 18 with Vite (exchange/src); page routing in App.jsx driven by lib/exchange-routes
Server	Node.js with Express (exchange/server/server.js), routes under /api
Database	PostgreSQL schema 'exchange' in production; SQLite for local development, selected by environment (tested)
Payments	Razorpay (order creation, signature verification, webhook); demo mode is refused against live keys (tested)
Identity	Hub single sign-on; a signed Exchange session token bound to the PINIT ID (tested)

Roles: one identity can be a seller (creator), a buyer, or both. Listing is gated by seller onboarding (a payment method must be verified) and buying is a separate capability that a creator enables.

18. Exchange Seller Workflow

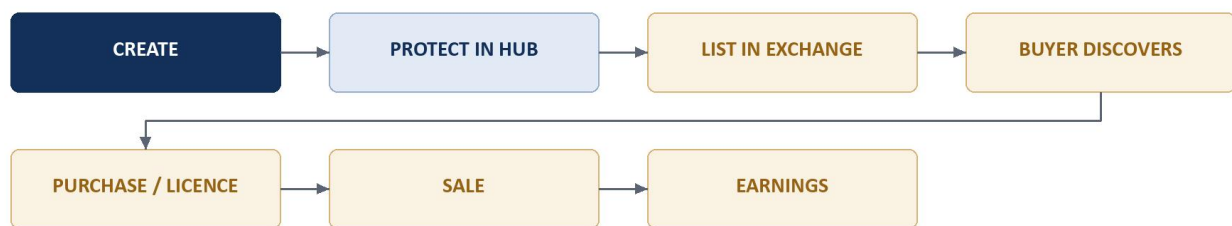


Figure 17. Seller journey

Stage	What happens	Screen	API	Records
Protected asset	The asset was protected in HUB.	HUB	HUB	Asset
List asset	Seller opens “List from Hub”, chooses a protected asset and sets tier prices.	ListFromHubModal	GET /hub/assets, POST /exchange/list-intent, POST /listings/from-hub	hub_assets, listings
Publish	HUB confirms the listing to Exchange; the listing becomes published.	SellerListings	POST /exchange/listings/confirm, POST /listings/:id/status	listings, hub_bridge_events
Buyer discovery	The listing is visible in the marketplace.	Marketplace, ListingDetail	GET /listings	listings
Purchase / licence	A buyer pays; the order is sealed.	CheckoutModal	POST /orders/checkout	orders_sealed, payment_intents
Sale	The seller sees the sale.	SalesCenter	GET /hub/seller-desk	orders_sealed
Earnings	Earnings are recorded per sale.	SellerEarnings	GET /hub/seller-desk	seller_earnings

Seller modules delivered: Overview, Assets, Listings, Sales, Earnings, List an Asset, and Opportunities (Section 19 covers Opportunities together with the buyer side of briefs).

Opportunities

Buyers post briefs; creators propose with work attached from their protected assets; the poster can invite creators, answer questions, form a team split that must add up to 100 percent and be accepted by each member, and award the brief exactly once. Creators can also post opportunities and collaboration requests. This module is covered by 23 passing tests.

19. Exchange Buyer Workflow

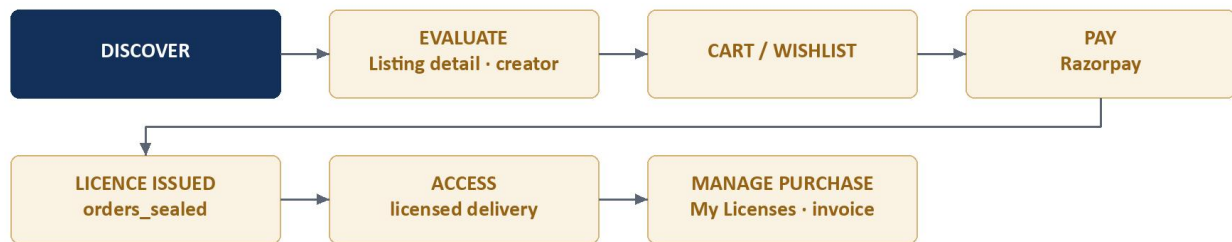


Figure 18. Buyer journey

Stage	What happens	Screen	API
Discover, search, filter	Browse and filter published listings.	HomePage, Marketplace	GET /listings
View asset and creator	Listing detail with the creator's profile.	ListingDetail, creator pages	GET /listings/:id, GET /creator/profile/:pinitId
Cart and wishlist	Save or collect listings.	CartPage, WishlistPage	/commerce/cart, /commerce/wishlist
Payment	Razorpay order created, paid, signature verified.	CheckoutModal	POST /orders/create-payment, POST /orders/verify-payment
Licence	The sale is sealed as an order with a licence; an exclusive licence locks the asset so a second sale fails.	(automatic)	POST /orders/checkout
Access	Licensed delivery through a HUB share link; download authorization checks buyer and refund status.	MyLicenses	POST /orders/download/authorize, POST /commerce/purchases/:sealId/share
Purchase record	Licences, orders, invoices and the certificate for a purchase.	MyLicenses, BuyerOrders, BuyerPayments	GET /orders/my-licenses, /my-orders, /invoice/:sealId, /certificate/:seal_id

Evidence

[O] 4 sealed orders (all licence and delivery status active) and 2 paid payment intents in production. [T] payment amount and order verification, listing lifecycle, exclusive-purchase locking, download authorization and idempotent bridge events pass. The automated end-to-end marketplace test currently fails at a role-gating assertion before reaching the purchase steps; see Section 26.

20. Exchange Database Architecture

Exchange uses its own schema (exchange) with 21 tables. Its tables are related by identifier values (pinit_id, asset_id, listing_id, seal_id); the schema declares only one database foreign key (seller_payment_methods.pinit_id to users). Integrity of the remaining relationships is enforced by the application.

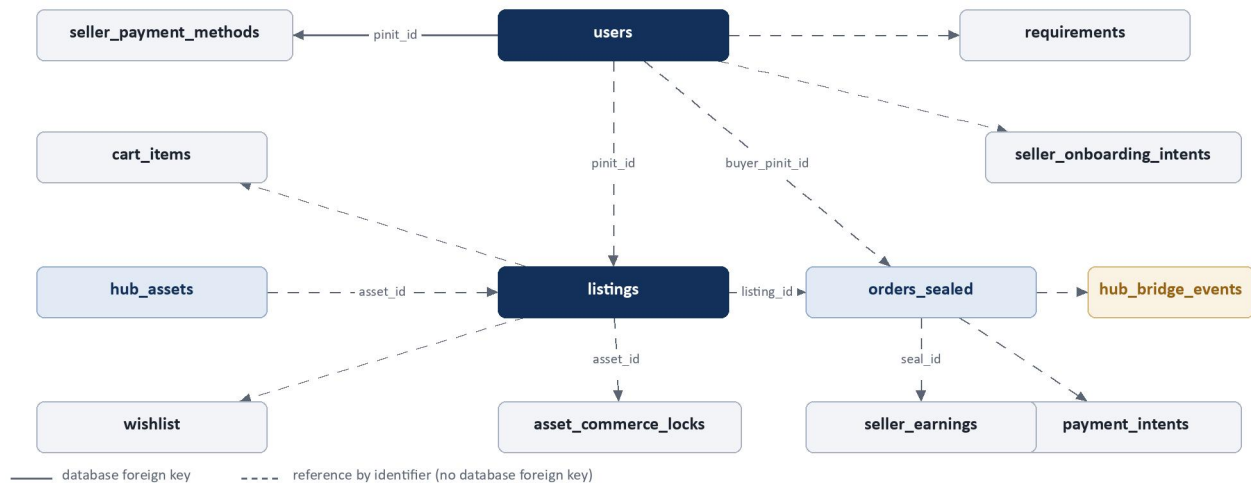


Figure 19. Exchange ER diagram

Table	Purpose	Primary key	Key references	Records in production
users	Sellers and buyers by PINIT ID	pinit_id	(root identity)	7
hub_assets	Public-safe copy of a HUB asset for listing	asset_id	asset_id = HUB Asset.id	20
listings	A listing with tier prices and status	listing_id	asset_id, seller pinit_id	6 (4 published, 2 unlisted)
orders_sealed	A sealed sale and its licence	seal_id	asset_id, listing_id, buyer_pinit_id	4
payment_intents	Razorpay payment attempts	id	order and buyer identifiers	2 (paid)
asset_commerce_locks	Exclusive-licence lock on an asset	asset_id	asset_id	0 (none yet exclusive)
hub_bridge_events	Idempotent Hub to Exchange events with retry	id	seal_id, asset_id	21
seller_earnings	Earnings per sale	id	seal_id, seller pinit_id	2
cart_items / wishlist	Buyer cart and saved listings	id	listing, buyer	4 / 3
creator_follows	Follows between users	id	creator, follower	1
seller_onboarding_intents / seller_payment_methods	Seller payment onboarding	id	pinit_id (payment methods: foreign key to users)	8 / 3
requirements and proposal tables	Briefs, proposals, invites, questions, teams	req_id, id	requirement, creator	3 briefs

Tables not treated as completed

reviews, coupons, refunds, disputes and payouts exist in the schema but hold no records and were not exercised; portfolio_profiles and tracking_jobs belong to the public ledger and monitoring bridge, whose tests currently fail or were not reviewed.

HUB to Exchange data path

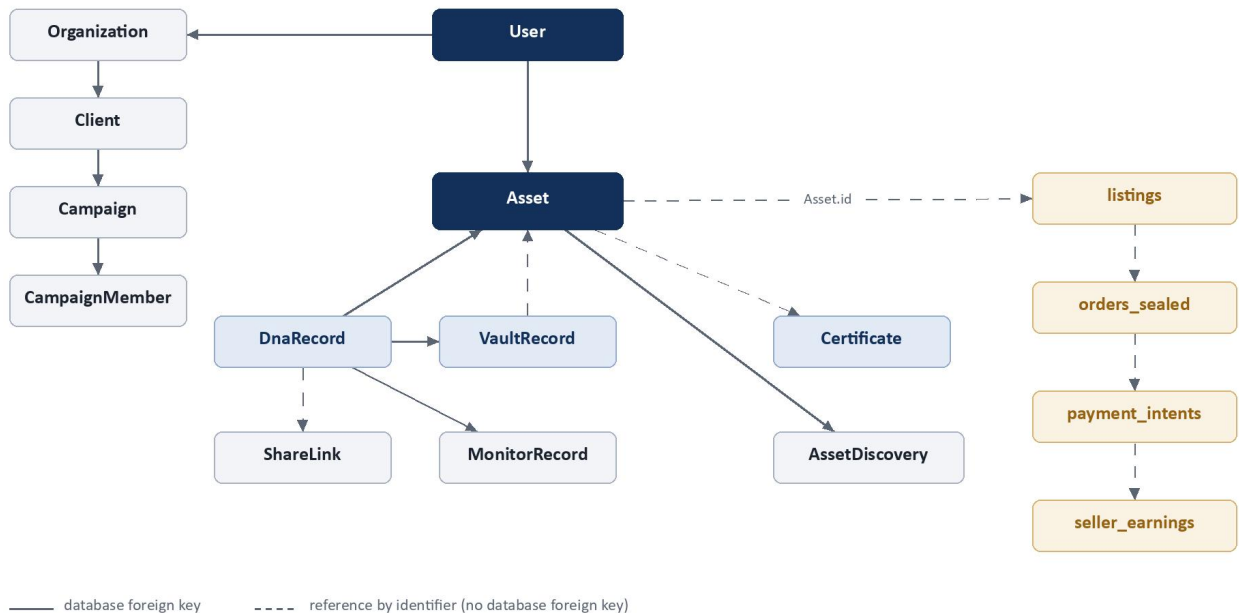


Figure 20. Entity map across HUB and Exchange

21. HUB + Exchange Integration

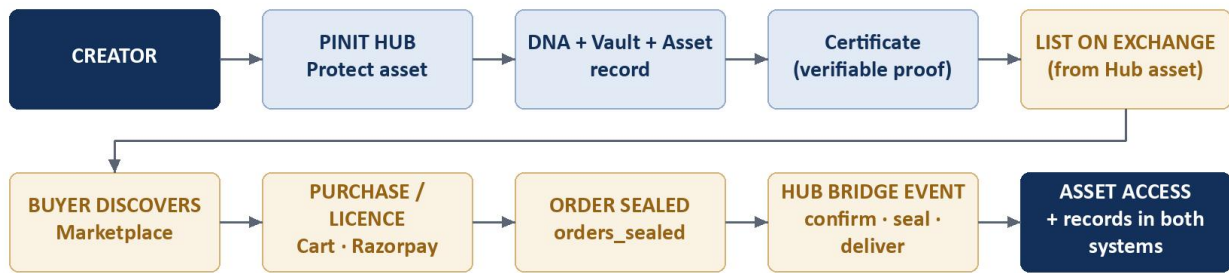


Figure 21. End-to-end integration flow

Implemented integration points

Integration point	How it works	Direction
Single sign-on	POST /exchange/sso returns a short-lived token; Exchange POST /auth/hub-sso exchanges it for an Exchange session bound to the PINIT ID.	HUB to Exchange
Listable assets	HUB exposes the owner's protected assets (GET /exchange/listable-assets, and a bridge variant secured by X-Pinit-Bridge-Secret).	HUB to Exchange
Listing intent and confirmation	POST /exchange/list-intent, then POST /exchange/listings/confirm; Exchange stores a public-safe hub_assets row and the listing.	Both
Protect from Exchange	POST /hub/protect-upload on Exchange forwards to HUB so a creator can protect a new file without leaving Exchange.	Exchange to HUB
Sale sealing	POST /exchange/sales/seal records the sale against the HUB asset.	Exchange to HUB
Licensed delivery	POST /exchange/share/create creates a HUB share link for the buyer; delivery is prepared and fetched through /exchange/delivery/*.	Exchange to HUB
Activity	POST /exchange/activity writes marketplace events onto the HUB asset timeline.	Exchange to HUB
Idempotent events	hub_bridge_events record each bridge event with retry metadata; replays are safe.	Exchange

Identifiers that connect the systems

Identifier	HUB	Exchange
PINIT ID	User.shortId	users.pinit_id (primary key)
Asset.id	Asset.id (canonical)	hub_assets.asset_id, listings.asset_id, orders_sealed.asset_id
Order / licence	appears on the asset timeline and delivery share link	orders_sealed.seal_id
Vault id, DNA id	HUB only. Never sent to Exchange; Exchange receives only public-safe data.	—

Security of the bridge: service-to-service calls carry a shared secret header; Exchange receives only public-safe listing payloads and short-lived sign-in tokens; vault encryption keys and full forensic DNA never leave HUB.

22. Frontend Architecture

Aspect	Pinit HUB (client/)	Pinit Exchange (exchange/)
Framework	React 18.3, TypeScript, Vite	React 18.3, JavaScript, Vite
Routing	react-router-dom 7; 65 route entries in router.tsx; RequireAuth guard on protected routes; nested DashboardLayout	Page routing in App.jsx using lib/exchange-routes (pageFromPath, pathForPage); role checks in lib/roles
State	React context (AuthContext, AccountViewModeContext) and hooks (useApi with cache keys, useSubscription, useOrganization)	Component state in App.jsx; lib/session (session token), lib/buyer
API communication	One axios instance with a JWT interceptor (services/dashboard.api); base URL from config/api.config	lib/api (apiFetch) with session token; /api rewritten to the Exchange server
Styling	Tailwind CSS 3, design tokens, light and dark themes (hooks/useTheme)	Own CSS (index.css)
UI building blocks	components/ui (Badge, Modal, EmptyState, Skeleton, TableWrap, PageShell); nav (Sidebar, Topbar)	components (ListingCard, CheckoutModal, ExchangeHeader, ProvenanceDrawer)
Notifications	react-hot-toast, notification inbox over server-sent events	toasts and account notification pages
Maps, PDF, QR	Leaflet, jsPDF, qrcode	Shared certificate component (via alias), Razorpay checkout

Layer	HUB	Exchange
Pages	client/src/pages (46 entries incl. business, home, subscription, profile)	exchange/src/pages (18 pages plus seller/, buyer/, info/)
Components	client/src/components	exchange/src/components
Services / API	client/src/services	exchange/src/lib
Hooks / state	client/src/hooks, context	exchange/src/hooks, lib/session

23. File and Project Structure

Simplified from the actual repository (folders not listed are omitted, not renamed).

```
Pinit-DNA/
  prisma/schema.prisma      111 models
  src/                      Hub backend (Express, TypeScript)
    api/{routes,controllers,middleware}
    services/               dna, vault, assets, certificates,
                           credentials, share, crawler, organization,
                           platform-events, subscription, exchange
    lib/ config/ validation/
  client/src/               Hub frontend (React, Vite)
    {pages,components,services,hooks,context,layouts,shared,lib}
  exchange/                 Pinit Exchange
    src/{pages,components,lib,hooks} frontend
    server/{routes,lib,schema,tests} API server and schema
  python-ai/                 FastAPI AI service (port 8001)
  scripts/                   ensure-*.cjs schema scripts
  tests/                     Hub Jest suites (120 committed)
  render.yaml                Hub backend service definition
```

24. End-to-End Data Flow

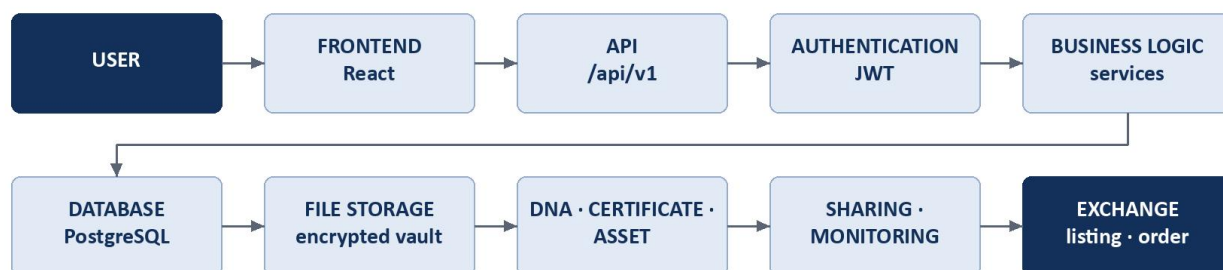


Figure 22. Overall system data flow

Transition	What moves
User to frontend	The user acts in the React application.
Frontend to API	HTTPS request to /api/v1 with a Bearer JWT.
API to authentication	requireAuth verifies the JWT; ownership middleware checks the resource owner.
Authentication to business logic	The controller calls a service with the authenticated owner id.
Business logic to database	Prisma reads and writes owner-scoped records.
Database and file storage	Encrypted file bytes go to Supabase Storage; metadata to PostgreSQL.
DNA, certificate, asset	Created in that order for a protected file (Section 6).
Sharing and monitoring	Links and monitor records reference the DNA record; accesses and scans append records and events.
Exchange and transaction	A HUB asset is listed through the bridge; a purchase is sealed in Exchange, and delivery is created back in HUB as a share link.

25. Completed Development Inventory

Every row is a capability that meets the definition of COMPLETED at the start of this document.

Area	Module	Implemented	Frontend	Backend	Database	Integration	Status
HUB · Identity	Password authentication and sessions	Sign-up, login, JWT + refresh, logout	pages/auth, AuthContext	/auth/*	User, RefreshToken, UserSession	All modules	COMPLETED
HUB · Identity	Face verification	Enrolment and login with liveness	components/auth	/auth/face/*	FaceTemplate, BiometricIdentity	Auth factor	COMPLETED
HUB · Identity	Passkeys	WebAuthn enrolment and login	lib/webauthn	/auth/passkey/*	WebAuthnCredential	Auth factor	COMPLETED
HUB · Workspace	Personal and Business accounts	Account type, business setup, switching	WorkspaceSwitcher	/auth, /organization	Organization, Workspace	Business	COMPLETED
HUB · Protect	Protect New	Upload to certified asset	GeneratePage	/dna, /vault, /certificates	DnaRecord, VaultRecord, Asset	Asset.id	COMPLETED
HUB · Protect	Pinit DNA, Layers 1 to 6	Six layers computed and stored	LayerPipeline	/dna/*	DnaRecord, layer records	Vault, monitoring	COMPLETED
HUB · Store	Vault, My Assets, Asset details	Encrypted storage, preview, rename, protected download	VaultPage, AssetsPage	/vault, /assets	VaultRecord, Asset	Sharing, Exchange	COMPLETED
HUB · Store	Security Check	Stored-file integrity check	VaultIntegrityPage	/vault/integrity-check	VaultRecord	Notifications	COMPLETED
HUB · Prove	Certificates	Issue, list, revoke, document with QR	CertificatesPage	/certificates, /credentials	Certificate, Credential	Asset, DNA, Vault	COMPLETED
HUB · Prove	Certificate verification	Public verification by id or link	VerifyCertificatePage	/certificates/verify	Certificate	Certificate document	COMPLETED
HUB · Share	Secure sharing	Links with access rules, revoke	VaultSharePage, ShareViewerPage	/share, /recipients	ShareLink, ShareRecipient	Business, Exchange delivery	COMPLETED
HUB · Share	Link intelligence	Access log, map, forward tree	AccessIntelligencePage	/share analytics, logs, tree	ShareAccessLog, LinkForwardEvent	Notifications	COMPLETED
HUB · Watch	Monitoring	Enrol, scan, match, alert	MonitoringPage	/monitor/*	MonitorRecord, CrawlResult	Asset discoveries	COMPLETED
HUB · Business	Clients and campaigns	CRUD, workspaces	ClientsPage, CampaignWorkspacePage	/business	Client, Campaign	Assets, people	COMPLETED
HUB · Business	People and access	Campaign members, per-person access	BusinessTeamPage	/business people	CampaignMember (Asset)	Sharing	COMPLETED
HUB · Business	Review, approval, handover	Versions, comments, approvals, handovers	CampaignWorkspacePage	/business versions, handovers	AssetVersion, VersionApproval	Sharing	COMPLETED
HUB · Business	Team, roles, departments	Invites, roles, audit log	OrganizationSettingsPage	/organization	OrganizationMember, Department	Business	COMPLETED
HUB · Portfolio	Portfolio	Build, publish, public page	ProfilePage	/portfolio	Portfolio*	Vault assets	COMPLETED
HUB · Account	Profile and sessions	Profile, avatar, sessions	ProfilePage	/profile	User, UserSession	Auth	COMPLETED
HUB · Account	Notifications	Inbox with live stream	notification bell	/notifications	Notification, PlatformEvent	All modules	COMPLETED

Area	Module	Implemented	Frontend	Backend	Database	Integration	Status
HUB · Account	Plans and entitlements	Plans, subscription, feature gating	UpgradePage	/subscription	Plan, Subscription	All gated modules	COMPLETED
Exchange · Market	Marketplace and listing detail	Browse, search, filter, view	Marketplace, ListingDetail	/listings	listings	HUB asset	COMPLETED
Exchange · Seller	Seller onboarding	Payment-method verification	SellerPaymentOnboarding	/seller/onboarding	seller_payment_methods	Razorpay	COMPLETED
Exchange · Seller	List from HUB	Choose protected asset, price, publish	ListFromHubModal	/hub, /listings/from-hub	hub_assets, listings	HUB bridge	COMPLETED
Exchange · Seller	Seller desk	Overview, assets, listings	pages/seller	/hub/seller-desk	listings	HUB	COMPLETED
Exchange · Seller	Sales and earnings	Sales list and earnings ledger	SalesCenter, SellerEarnings	/hub/seller-desk	orders_sealed, seller_earnings	Orders	COMPLETED
Exchange · Seller	Opportunities	Briefs, proposals, invites, awards	SellerOpportunities	/opportunities	requirements, proposals	Creators	COMPLETED
Exchange · Buyer	Cart and wishlist	Save and collect listings	CartPage, WishlistPage	/commerce	cart_items, wishlist	Checkout	COMPLETED
Exchange · Buyer	Checkout, payment, licence	Razorpay payment, sealed order, exclusive lock	CheckoutModal	/orders	payment_intents, orders_sealed	HUB bridge	COMPLETED
Exchange · Buyer	My Licenses and orders	Licences, orders, invoices, certificate	MyLicenses, BuyerOrders	/orders	orders_sealed	HUB delivery	COMPLETED
Integration	HUB to Exchange bridge and SSO	SSO, listing confirm, sale seal, delivery, activity	ListFromHubModal	HUB /exchange, Exchange /hub	hub_bridge_events	Asset.id, PINIT ID	COMPLETED

26. Testing and Validation

Area	Evidence in the reviewed implementation
Unit and integration (Hub)	Jest with ts-jest: 120 committed suites under tests/ (plus 8 added locally for uncommitted work, which are included in the run below) covering authentication and biometrics, DNA layers, vault, sharing, notifications, portfolio, credentials, tenant isolation and authorization boundaries.
Last complete Hub run	1,072 tests passed, 8 failed, 23 skipped across 119 of 128 suites run. Two failures (credentials-phase1) were then fixed and re-verified; the full suite was not re-run after that fix.
Known Hub failures	Four forensic suites (unified-investigation, enterprise-investigation-pipeline, candidate-ranking-engine, evidence-pairing; 6 tests) and health.test.ts, which cannot compile because the supertest dependency is not installed.
Exchange	node:test, 97 tests: 93 passed, 4 failed. Passing areas: database selection, session and preview tokens, payment verification and demo-mode guard, opportunities (23 tests), listing lifecycle, exclusive-purchase locking, download authorization, bridge-event idempotency. Failing: the end-to-end marketplace flow (stops at a role-gating assertion), two role-policy tests, and one public-ledger field test.
Authentication and security	Face liveness and matching, passkeys, cookies, JWT subject and isolation, no raw biometric data in logs or responses, authorization boundaries, tenant isolation: all passing.
Database	Verified by read-only queries against production for the row counts quoted in this document. No migration test suite exists; schema changes are applied by idempotent scripts.
Frontend	Testing evidence not available in the reviewed implementation. The client has no test runner configured; one stray unit test file exists.
Deployment	Testing evidence not available in the reviewed implementation. There is no CI workflow in the repository.

Reading these numbers

The failing forensic suites are why investigation is excluded from the completed scope. The failing Exchange end-to-end test is on a role-gating expectation that later behaviour contradicts (a passing test states that creators and buyers may share one identity); marketplace purchases nevertheless exist in production. The test needs updating; until it does, the purchase flow is supported by operation and component tests rather than by a passing end-to-end test.

27. Deployment Architecture

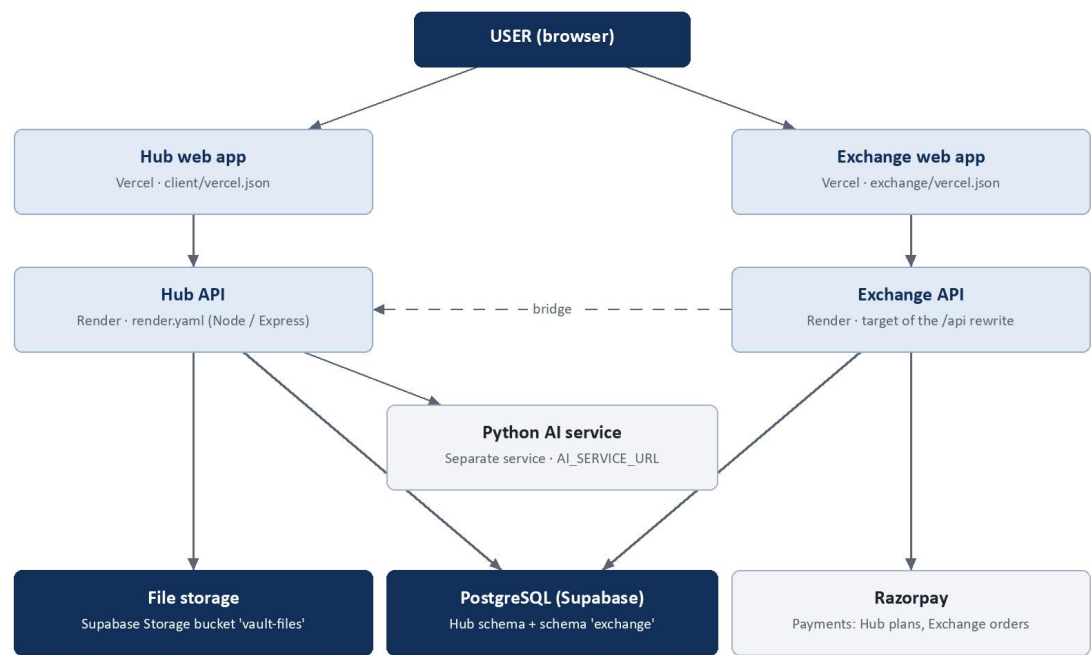


Figure 23. Deployment architecture

Element	Configuration found in the repository
Hub web app	Vercel (client/vercel.json): Vite build, single-page rewrites
Exchange web app	Vercel (exchange/vercel.json): Vite build; /api rewritten to the Exchange server on Render; share paths forwarded to the Hub domain
Hub API	Render web service (render.yaml): npm run build, then start:prod, which applies the ensure-*.cjs schema scripts and starts the server
Exchange API	Render service (target of the Exchange /api rewrite)
AI service	Separate service (python-ai, FastAPI, Dockerfile); reached through AI_SERVICE_URL
Database	Supabase PostgreSQL; Hub schema and the exchange schema
File storage	Supabase Storage bucket vault-files
Payments	Razorpay
Environment	Render environment variables (secrets not in the repository); .env.example files document the variables
Deployment process	Git push to the deploy branch triggers Render and Vercel builds. There are no CI workflow files.

The live status of these services was not re-verified on the documentation date; this section describes the configuration in the repository.

28. User Journeys

Individual user

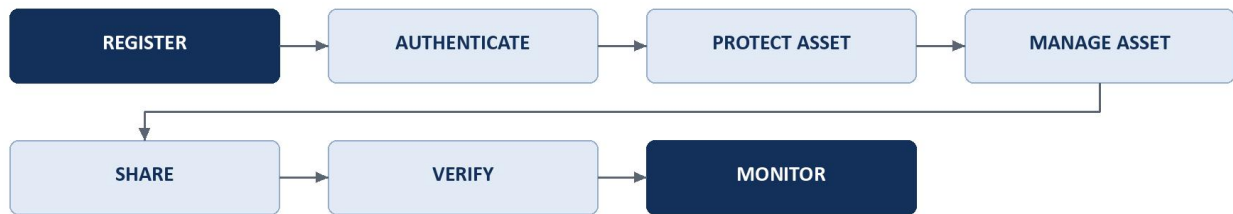


Figure 24. Individual user journey

Business user

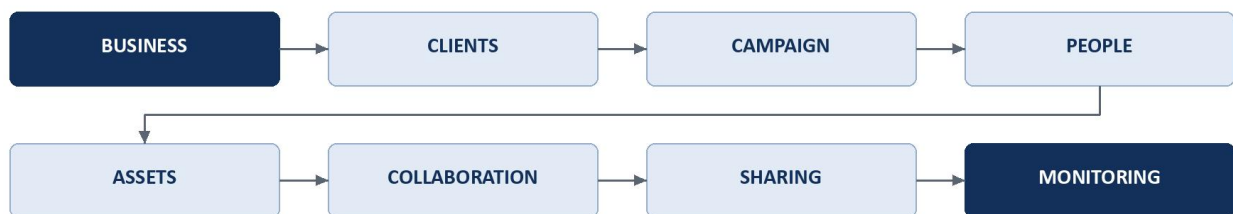


Figure 25. Business user journey

Creator / seller

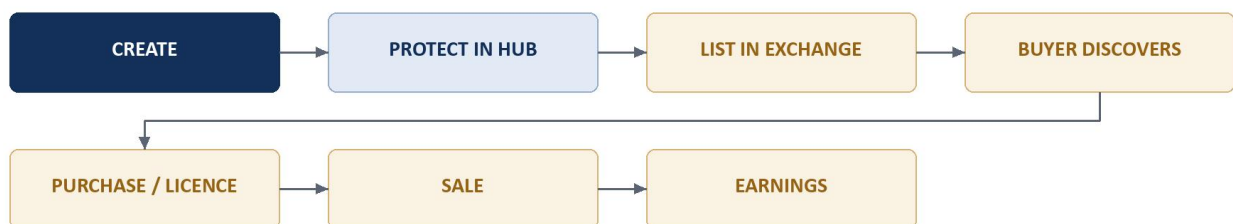


Figure 26. Creator and seller journey

Buyer

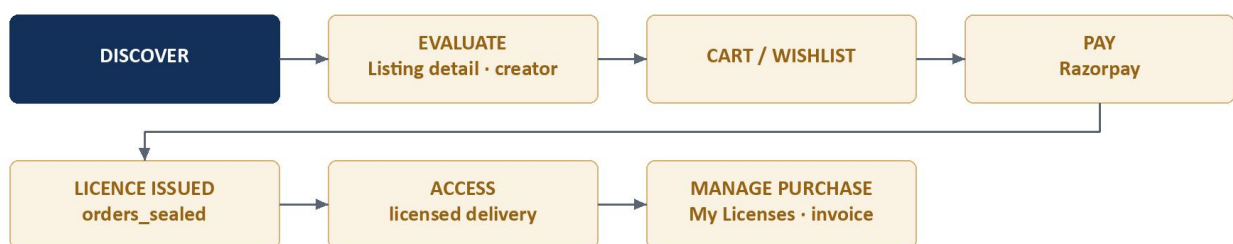


Figure 27. Buyer journey

29. Technical Entity Map

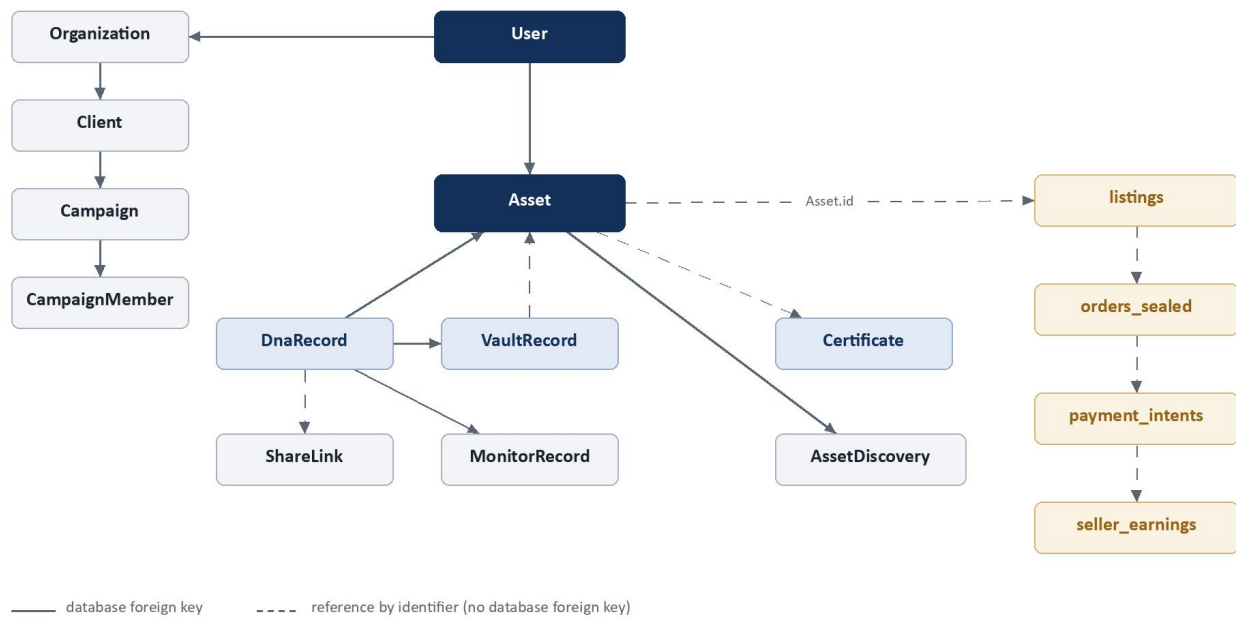


Figure 28. Consolidated entity map (implemented entities only)

The map joins the HUB entities to the Exchange tables through Asset.id. The Exchange tables are related to one another by identifier values (Section 20). Entities named in the brief that are not implemented as separate entities are not shown: there is no separate “Finding”, “Investigation” or “License” entity in the completed scope (a licence is the order_sealed record).

30. Completed Product Capabilities

Pinit HUB

- Identity: password, face verification with liveness, passkeys, sessions
- Asset protection and digital asset identity (Asset.id)
- Pinit DNA (Layers 1 to 6) and an ownership watermark
- Encrypted protected storage (Vault) and asset management
- Certificates and public verification, with revocation and expiry
- Secure sharing with access controls, activity tracking and link intelligence
- Monitoring of protected assets across supported sources
- Business Workspace: clients, campaigns, people, review, approval, handover
- Portfolio, notifications, plans and entitlements

Pinit Exchange

- Marketplace discovery, listing detail, cart and wishlist
- Seller onboarding and listing from a protected HUB asset
- Buyer checkout with Razorpay, sealed licence records, invoices, licensed delivery
- Sales and earnings for sellers
- Opportunities: briefs, proposals, invitations, teams and awards

Integration

- Single sign-on, listable assets, listing confirmation, sale sealing, licensed delivery and activity between HUB and Exchange
- Idempotent bridge events with retry

31. Final Development Status

What Has Been Delivered

As of the documentation date, the following has been fully developed and evidenced:

- **Pinit HUB:** a complete protect-to-verify path (identity, DNA, encrypted Vault, canonical Asset, signed Certificate), secure sharing with tracking and revocation, monitoring of supported sources, a Business Workspace, a Portfolio, notifications and plans. Backed by a 111-model schema and 120 committed test suites, with production data showing 42 assets, 32 certificates, 39 share links and 96 monitoring profiles.
- **Pinit Exchange:** a working marketplace with seller onboarding, listing from HUB assets, cart and wishlist, Razorpay checkout, sealed licences, licensed delivery, sales, earnings and opportunities. Production data shows 6 listings, 4 sealed licences and 2 paid payment intents.
- **Integration:** a verified bridge that carries one asset identity, one user identity and licensed delivery between the two products.

This document represents the implemented product state as of the documentation date.

Not Included in Completed Scope

These items exist in the repository or were developed, but do not meet the definition of COMPLETED. They are listed so that the boundary is explicit.

Item	State	Reason it is not counted
Investigation and evidence reporting (unified investigation, forensic comparison, evidence PDFs, business investigations)	Implemented	Four forensic test suites fail in the reviewed tree; no incident or evidence records exist in production.
Asset Tracking pages and the redesigned Asset Activity lifecycle log	Developed and checked locally	Not committed or deployed. The committed Asset Activity screen is superseded.
Unified lifecycle event layer and compact video-frame DNA storage	Developed and tested locally	Not committed. The lifecycle columns have not been applied to the production database.
Certificate v2 binding (asset id and file hash in the signature), restricted public verification view, evidence-record wording on the certificate, configurable public verification origin, shared certificate component across HUB and Exchange	Developed and tested locally	Not committed or deployed. Until deployed, the limitations listed in Section 14 apply.
Passkey use in production	Implemented and tested	No enrolment is recorded; included in the inventory on test evidence only.
Hub plan payment collection (Razorpay for subscriptions)	Implemented	A demonstration completion path exists and live collection was not verified.
Organization API keys, webhooks and third-party integrations (Slack, Teams, Zapier, Dropbox, Google Drive)	Implemented	No evidence of use or tests.
Exchange reviews, coupons, refunds, disputes, payouts	Schema and routes	No records; not exercised.
Exchange public creator ledger and portfolio pages	Implemented	A test on which public fields may be published fails.
Additional DNA layers and modes (further layer tables, DNA vNext, block DNA), spatial pixel authentication, Publish Guardian and browser extension, Master Admin console	Present in the codebase	Outside the verified scope of this document.

Documentation Scope

This document represents the functionality verified as implemented in the reviewed Pinit HUB and Pinit Exchange codebase as of 21 September 2026. Features that were incomplete, experimental, planned, disabled, or not sufficiently verified have not been represented as completed functionality.

Endpoints documented in Section 11: 287 of 384 route registrations detected in the reviewed route files.